

惟安数据科技（北京）有限公司 WEIAN DATA TECH

IRTC

使用手册

中文·零基础完整版

从一张表格到一份读得懂的报告——项目反应理论（IRT）分析全流程

V1.1.2 正确性修正版 · 已发布于 CRAN

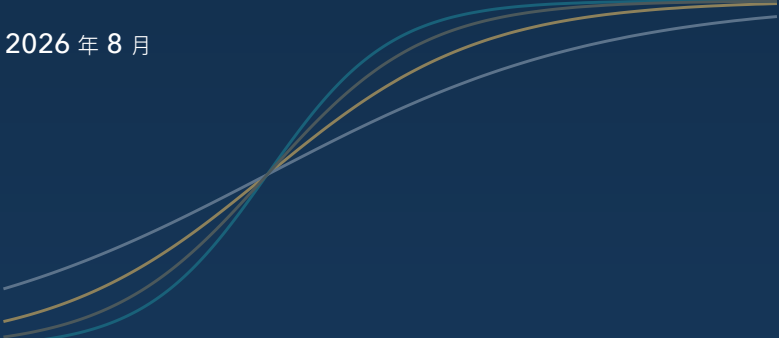
R 包制作人 / 维护人：马崑翔

Kunxiang Ma · makunxiang@weiansdata.com

著作权人：惟安数据科技（北京）有限公司

软件版本 1.1.2 (2026-08-27) · 手册发布 2026 年 8 月

`install.packages("IRTC")`



版本与版权信息

手册名称	IRTC 使用手册（中文 · 零基础完整版）
对应软件	IRTC 1.1.2，发布日期 2026-08-27
发行渠道	CRAN (<code>install.packages("IRTC")</code>) ； 源 码 仓 库 GitHub weandata/IRTC
制作与维护	马崑翔（Kunxiang Ma）， makunxiang@weandata.com
著作权人	惟安数据科技（北京）有限公司， contact@weandata.com
软件许可	GPL (≥ 2)；详见软件包内 LICENSE 与 <code>inst/COPYRIGHTS</code>
方法依据	Adams, Wilson & Wang (1997), <i>Applied Psychological Measurement</i>
手册版本	V1.1.2，2026 年 8 月

关于版本一致性 本手册与 IRTC 1.1.2 逐条核对。**1.1.2** 是正确性修正版：其中三处修正涉及 1.1.1 及更早版本中会静默产生错误结果的输出——多维流式引擎报告的对数似然与信息准则、多维模型的题目参数表，以及答案键实际作用的选项。若你用早期版本做过分析，请先读附录 D。若手册与包内帮助页（`?irtc`、`?irtc.mml`）出现不一致，以帮助页为准。

引言

做过问卷和测验的人都熟悉这样一张表：每一行是一个人，每一列是一道题。把它变成结论——哪道题出得好、每个人水平多高、不同人群有没有差异——中间隔着的往往不是统计知识，而是一整套工具链。

现有的 IRT 软件包默认使用者是统计程序员：数据要先整理成干净的数值矩阵，结果要自己从模型对象里取出来再排版。IRTC 想把这一头一尾也接上，同时完整保留底层的专业接口——数据进得来，结果出得去，算得快也算得大。

这本手册写给四类读者

调查与测评人员——不需要任何编程或统计基础，从「电脑上什么都没装」开始，一步步装好软件、整理数据、跑出结果，并读懂每一个数字。

统计人员——完整的专业接口：潜在回归、多组、多维、固定参数、抽样权重，以及三套可选的估计引擎。

决策者——三种受众版式的 Word/HTML 报告，结论先行。

AI 助手与自动化流水线——结构冻结的机器可读结果与带错误码的结构化报错。

怎么读这本手册？ 只想最快出结果，直接翻第 4 章，十分钟从 Excel 到成品报告；完全的新手请从第 0 章顺序读到第 6 章，照做一遍即可独立完成一次分析；会用电脑但不懂 IRT 的读者，重点读第 1 章、第 5 章和第 6 章；遇到报错查第 8 章，忘了名词查第 9 章，找函数用法查第 14 章。每章末尾都有「本章你应该掌握的」小结，用来确认自己有没有读懂。

关于这一版。 IRTC 已通过 CRAN 审核并正式上架，在 R 中运行 `install.packages("IRTC")` 即可安装。1.1.2 是一次正确性修正版：它源于一次真实的全国性调查分析，以及随后用原始数据文件做的全量验证，共修正 15 处缺陷，其中三处会静默产生错误结果。题目参数与个人能力估计 不变，变的是此前算错的那些输出。升级须知见附录 D。

IRTC 制作人 / 维护人 · 惟安数据科技（北京）有限公司 · 2026 年 8 月

目录

关于本手册	1
为什么选择 IRTC (核心亮点)	1
这本手册怎么读	2
0 最最基础：在开始之前	3
0.1 我们要解决一个什么样的现实问题？	3
0.2 你需要准备什么？	3
0.3 几个你会反复听到的词（先混个眼熟，第 9 章有详细解释）	3
1 先把概念讲透（零统计基础）	5
1.1 测验、问卷、量表到底在测什么？	5
1.2 为什么「把分数加起来」不够好？	5
1.3 看不见的 θ ，怎么测？——用「概率」搭桥	6
1.4 题目的两个「性格」：难度与区分度	6
1.5 二分题 vs 多级题	7
1.6 单维 vs 多维	7
1.7 IRTC 能做和不能做什么	7
2 准备你的电脑（零编程基础）	8
2.1 R 和 RStudio 是什么？为什么要两个？	8
2.2 第一步：安装 R	8
2.3 第二步：安装 RStudio	8
2.4 认识 RStudio 的界面（四块区域）	9
2.5 第一次运行命令：把控制台当计算器	9
2.6 第三步：安装 IRTC 包	10
2.7 第四步：加载包	11
2.8 跑通第一个完整例子（用软件自带数据）	11
3 把数据交给 IRTC（大部分整理是自动的）	12
3.1 你的文件长什么样就行	12
3.2 IRTC 自动帮你做的清洗	12
3.3 多级计分（Likert 等）——现在是自动的	13

3.4	缺失值——自动转换，但要认识NA	13
3.5	反向计分题——这一步仍需你人工处理!	14
3.6	原始 A/B/C/D 作答——用答案键自动计分	14
3.7	分析前体检: <code>irtc_check_data()</code>	16
3.8	常见数据问题: 现在谁来管?	16
3.9	多维分析必备: Q 矩阵怎么写?	16
3.10	背景变量、分组、权重怎么单独存?	17
4	核心用法: 从一行命令到完整交付	19
4.1	主入口: <code>irtc()</code> 一行命令	19
4.2	白话结果: <code>plain_summary()</code> 与质量评级	21
4.3	交付三件套: Excel、报告、图	21
4.4	语言切换与 AI/自动化接口	22
4.5	<code>irtc.mml()</code> : 固定斜率模型 (Rasch / PCM / RSM)	22
4.6	<code>irtc.mml.2pl()</code> : 含区分度模型 (2PL / GPCM)	24
4.7	<code>method</code> : 三个引擎到底怎么选?	25
4.8	<code>control</code> : 控制参数全清单 (逐项解释)	25
4.9	查看结果的 4 个方法 (S3 方法)	27
5	读懂结果 (每个输出讲透)	29
5.1	三条读结果的路径	29
5.2	题目参数表 <code>m\$xsi</code> 怎么读?	29
5.3	区分度怎么看? (仅 2PL/GPCM)	30
5.4	个人能力分 <code>m\$person</code> 怎么用?	30
5.5	潜在分布 <code>m\$variance</code> / <code>m\$gmean</code> 怎么读?	31
5.6	模型比较: 对数似然、AIC、BIC	32
5.7	加速后必看: <code>m\$accuracy_report</code>	32
5.8	路由信息: <code>m\$routing</code>	33
5.9	信度与测量精度: 我的问卷「准」吗?	33
5.10	怎样判断「好题 / 坏题」并决定删改?	34
5.11	把 EAP 能力分接回你的原始数据	34
6	完整实战案例 (从文件到结论)	36
	案例 A: 一份 0/1 的能力测验 (最常见)	36
	案例 B: 原始 A/B/C/D 作答 + 答案键	36
	案例 C: 5 点 Likert 满意度问卷 (含反向题)	37
	案例 D: 多维 (两个分量表)	37
	案例 E: 大数据 + 受控精度加速 + 误差核查	37
	案例 F: 分组比较 (如男女差异)	38
	案例 G: 跨年度题库链接 (参数表的用法)	38
7	数学定义与原理 (为没学过统计的人解释)	39
7.1	概率: 先统一一个最基本的词	39

7.2	逻辑斯蒂曲线：IRT 的「S 形」核心	39
7.3	似然：「哪套参数最像我看到的数据？」	40
7.4	把能力「积分掉」：边际似然（MML）	40
7.5	EM 算法：先猜能力、再修题目，来回迭代	40
7.6	多维与「维度灾难」	40
7.7	流式引擎：为什么能算百万人	41
7.8	受控精度：用「预算」换速度，用「核查」保安全	41
7.9	模型识别：为什么要「固定一些东西」	41
7.10	信息准则：怎么公平比较模型	42
8	全量报错解决手册	43
8.1	数据与参数类	44
8.2	引擎与路由类	45
8.3	数值与收敛类	46
8.4	安装与编译类	46
9	术语表（中英对照）	48
10	推荐参考文献与书目	51
11	IRTC 的技术路线、理论基础与优化效果	53
11.1	总体技术路线	53
11.2	精确性与稳定性原则	54
11.3	已达到的效果	54
12	常见问题解答（FAQ）	55
13	把结果写进报告的模板	57
13.1	方法部分（Methods）模板	57
13.2	题目质量表（建议放附表）	57
13.3	信度与精度	58
13.4	人群比较 / 潜在回归	58
13.5	加速与可信度（若用了 fast）	58
13.6	一句话写「为什么用 IRT 而不是总分」	58
14	V1.1.2 全函数参考（速查 + 详解）	59
14.1	全函数速查表	59
14.2	一站式与数据层	61
14.3	统计层	63
14.4	估计层（专家 API，与 0.1.0 完全兼容）	63
14.5	输出层	63
14.6	AI 与自动化层	64

附录	66
附录 A：一页速查卡	66
附录 B：CSV 数据模板（二分题）	67
附录 C：新手常用 R 基础操作	67
附录 D：版本与更新记录	67

关于本手册

本手册对应 IRTC 1.1.2 (2026-08-27)。在 R 里一条命令即可安装：`install.packages("IRTC")` (见第 2.6 节)。加载后包会打印版本横幅 * IRTC 1.1.2 (2026-08-27)，可据此确认你装的是本手册对应的版本。

1.1.2 是正确性修正版，如果你用过更早的版本，请先看附录 D。它修正了 15 处缺陷，其中三处会静默产生错误结果：多维流式引擎报告的对数似然与 AIC/BIC 有系统性偏移、多维模型的题目参数表只读第 1 维、以及答案键可能作用在错误的选项上。用过 `key=` 计分或做过多维模型比较的分析，请按附录 D 的方法排查。

为什么选择 IRTC (核心亮点)

市面上的 IRT 软件包 (TAM、mirt、ltm 等) 都默认你是统计程序员：数据要自己整理成规范的数值矩阵，结果要自己从模型对象里抠出来再排版。IRTC 把「数据进来」和「结果出去」这两头也做完了：

- 数据进得来 (输入优势)：直接读取 Excel (.xlsx/.xls)、CSV/TSV (自动识别分隔符和 UTF-8/GBK 编码)、SPSS (.sav)、Stata (.dta)、SAS (.sas7bdat) 和 R 数据框；自动分离学号/编号列、自动把缺失码 (-9、99 等) 转成缺失、自动把 A/B/C/D 原始作答按答案键计分。每一步清洗都记入日志，可追溯。
- 结果出得去 (输出优势)：一条命令生成三个可直接交付的 Excel 文件 (红黄绿配色的题目质量表、跨年度链接用的参数表、可整体粘贴的能力值表)，以及面向决策者/调查人员/统计人员三种版式的 Word/HTML 分析报告，全部中英双语可切换。
- 算得快 (速度优势)：估计核心是 Rcpp/C++ 并行实现，配 SQUAREM/超松弛加速；可选的「受控精度」模式裁剪低权重积分节点并附上实测误差报告 (精确计算始终是默认)。
- 算得大 (规模优势)：独有的有界内存 streaming 引擎不在内存中展开「人数 × 节点数」后验矩阵，普通笔记本即可标定百万级作答的题目间多维模型；引擎按数据规模自动路由，无需手工选择。
- 两套接口，四类人群：零基础人员用一行 `irtc("数据.xlsx", model="1PL")`；统计人员用与 TAM 同风格的全参数 `irtc.mml()` 系列 (潜在回归、多组、多维、固定参数、抽样权重一应俱全)；决策者拿报告；AI 助手拿机器可读结果。
- 对 AI 与流水线友好：`irtc_results()/irtc_json()` 输出结构冻结的 tidy 数据；所有报错带错误码、原因、修复建议三个机器可读字段；包内自带 `llms.txt` 供 AI 助手一次读入——这是首批为自动化调用方专门设计接口的 IRT 包。

- 正式发布、随手可得：已通过 CRAN 审核并上架，`install.packages("IRTC")` 即可安装，Windows/macOS 用户直接拿到编译好的二进制包，不需要自己配置 C++ 编译环境。
- 复杂抽样调查友好：抽样权重不只进入 IRT 估计，经典难度、区分度、题目拟合、质量评级、百分位与 T 分数全部按加权口径计算，一张结果表不会出现「IRT 加权、经典统计未加权」的混用口径（见 4.1）。
- 可回溯核查：`irtc_audit_scoring()` 能直接判定一份存档分析是否受早期版本的计分缺陷影响，不必重跑（见 3.6）。
- 质量可验证：1298 项自动化测试，总体代码覆盖率 96%、关键模块 $\geq 95\%$ ，R CMD check --as-cran 0 错误 0 警告 0 提示，三大平台（macOS/Windows/Linux）持续集成。

急着上手？直接看第 4 章（一行命令到完整交付），函数速查表在第 14 章。

这本手册假设你：

- 没写过任何代码，没用过 R；
- 没系统学过统计，听到「似然」「方差」会发怵；
- 但你手里有一份问卷 / 测验数据（比如 Excel 表），想知道「每道题好不好」「每个人水平多高」「不同人群有没有差异」。

这本手册保证：从「电脑上什么都没装」开始，一步一步带你装好软件、把数据弄成正确格式、跑出结果、并读懂每一个数字的含义。遇到报错，你可在本手册里查到怎么修。所有专业名词第一次出现都会用大白话解释，并在最后的术语表里再解释一遍。

这本手册怎么读

- 只想最快出结果：直接读第 4 章前半（一行命令到完整交付），十分钟从 Excel 文件到成品报告；函数细节查第 14 章速查表。
- 完全新手：从第 0 章一直读到第 6 章，照着做一遍，你就能独立完成一次分析。第 7 章（原理）可以略读，想深入再回来。
- 会用电脑、只是不懂 IRT：重点读第 1 章（概念）、第 5 章（读结果）、第 6 章（案例）。
- 遇到报错：直接跳到第 8 章（报错手册）按信息查。
- 忘了某个词：查第 9 章（术语表）。

每一章末尾有「本章你应该掌握的」小结，帮你确认有没有读懂。

最基础：在开始之前

0.1 我们要解决一个什么样的现实问题？

设想你是一名调查工作人员。你做了一份问卷或一场考试，收回了很多份答卷。现在你面前是一张大表格：每一行是一个人，每一列是一道题，格子里是这个人在这道题上的作答（比如对/错，或者「非常不同意 = 1 … 非常同意 = 5」）。

你想回答这样一些问题：

1. 这些题目本身好不好？哪道题太难、哪道题太简单？哪道题能很好地区分高水平和低水平的人，哪道题谁来做都差不多（没有区分力）？
2. 每个被调查者的「真实水平」是多少？不是简单地数对了几道题，而是一个更科学的、考虑了「题目难易不同」的能力分数。
3. 不同人群有没有差异？比如男生女生、城市农村、实验组对照组，他们的平均水平、分布是否不同？
4. 能不能用一些背景信息（年龄、教育程度等）来解释能力？

IRT 这个软件包，就是用来回答上面这些问题的工具。它背后用的方法叫项目反应理论（Item Response Theory，简称 IRT）。

0.2 你需要准备什么？

- 一台电脑（Windows 或 Mac 都行）。
- 你的数据（最好是 Excel 或 CSV 文件，第 3 章会教你怎么整理）。
- 大约 1~2 小时，跟着本手册走一遍。
- 不需要任何编程或统计基础——本手册会从零教起。

0.3 几个你会反复听到的词（先混个眼熟，第 9 章有详细解释）

词	一句话理解
R	一个免费的统计软件（就像一个超强的计算器 + Excel）。

词	一句话理解
R 包 (package)	给 R 增加新功能的「插件」。IRTC 就是一个包。
函数 (function)	一条「做某件事」的命令，比如「估计模型」。你给它原料（数据），它给你结果。
参数 (argument)	给函数的「设置选项」，比如「用哪种模型」「数据是哪个」。
作答矩阵 / 响应矩阵 (response matrix)	就是那张「人 × 题」的大表格。
能力 / 潜在特质 (θ , theta)	我们想测但看不见的东西，比如「数学水平」「满意度」。
题目参数	描述一道题「性格」的数字，主要是难度和区分度。

本章你应该掌握的：你大概知道了 IRTC 是干嘛的（分析问卷/测验数据），以及你需要准备一台电脑和一份数据。下面我们正式开始。

先把概念讲透（零统计基础）

这一章不碰电脑，只讲道理。读懂了，后面操作才不会「知其然不知其所以然」。

1.1 测验、问卷、量表到底在测什么？

当你做一份数学测验，你真正关心的不是「这个人这 20 道题对了几道」，而是这个人的数学水平到底有多高。「数学水平」是一个看不见、摸不着的东西——我们叫它潜在特质或潜在能力，在公式里通常用希腊字母 θ （读作「西塔」，**theta**）表示。

同理：

- 一份抑郁量表测的是看不见的「抑郁程度」；
- 一份满意度问卷测的是看不见的「满意度」；
- 一场英语考试测的是看不见的「英语水平」。

题目（**item**）只是一个「窗口」：我们通过一个人在很多道题上的表现，去间接推断那个看不见的 θ 。这就是 IRT 的根本思路。

1.2 为什么「把分数加起来」不够好？

最朴素的做法是总分：数对了几道题，或者把 Likert 各题（1~5 分）加起来。这叫经典测量理论（**CTT**）。它简单，但有几个大问题：

问题一：每道题难度不一样，但总分一视同仁。假设甲做对了 5 道很难的题，乙做对了 5 道很简单的题。总分都是 5，但显然甲的水平更高。总分法看不出这个差别，IRT 能。

问题二：分数依赖于「这套题」，换一套题就不可比。用难卷子考出来的 60 分，和用容易卷子考出来的 60 分，不是一回事。IRT 把题目的难度和人的能力放在同一把尺子上，使得「人」的估计不依赖于具体用了哪套题（这叫「参数不变性」，是 IRT 最重要的优点之一）。

问题三：总分无法告诉你「这道题好不好」。IRT 会给每道题估计出「难度」「区分度」，让你能挑出坏题、改进问卷。

一句话：CTT 关心「总分」，IRT 关心「题目和人各自的性质，并把它们放在同一把尺子上」。这就是为什么大型测评（高考类、PISA、国际比较、心理量表标准化）几乎都用 IRT。

1.3 看不见的 θ ，怎么测？——用「概率」搭桥

关键的桥梁是一句话：

一个人在一道题上答对（或选高分）的概率，取决于「他的能力 θ 」和「这道题的性质」。

直觉上很自然：

- 能力 θ 越高 $\rightarrow$ 答对的概率越大；
- 题目越难 $\rightarrow$ 答对的概率越小。

IRT 把这句直觉写成了精确的数学公式（第 7 章会展开，这里只讲意思）。有了公式，我们就能反过来做一件神奇的事：

我们看得见每个人在每道题上的作答（数据），于是可以反推出那些看不见的东西——每个人的 θ 、每道题的难度和区分度。这个「反推」的过程叫估计（estimation）。

1.4 题目的两个「性格」：难度与区分度

每道题最重要的两个性质：

① 难度（difficulty，记作 b ）

- 通俗说：这道题有多难答对（或多难选高分）。
- 难度高的题，只有高能力的人才答得对；难度低的题，大部分人都答得对。
- 在 IRT 里，难度和能力用同一把尺子衡量：如果一个人的能力 θ 正好等于某题的难度 b ，那他在这道题上答对的概率是 50%。
- 数值通常在 -3 到 $+3$ 之间：负数 = 容易，正数 = 难， 0 = 中等。

② 区分度（discrimination，记作 a ）

- 通俗说：这道题「分辨能力高低」的本事有多强。
- 区分度高的题：高能力的人几乎都对，低能力的人几乎都错——一道题就能把人分开，是「好题」。
- 区分度低（接近 0 ）的题：不管能力高低，答对概率都差不多——这道题「没用」，谁来都一样，应该改掉。
- 数值通常在 0.5 到 2.5 之间：越大，区分力越强。注意：只有 2PL/GPCM 这类模型才估计区分度；Rasch/1PL 假设所有题区分度都一样（都等于 1 ）。

生活类比：把「能力 θ 」想成「跳高水平」，把「题目难度 b 」想成「横杆高度」。横杆越高（题越难），跳过去越难。区分度 a 则像「横杆判定的灵敏度」：高区分度的横杆，差一点就判失败、够了就判成功，泾渭分明；低区分度的横杆，跳没跳过都模模糊糊。

1.5 二分题 vs 多级题

- 二分题 (**dichotomous**): 只有两种结果, 对/错、是/否, 编码成 0/1。例: 选择题判对错。
- 多级题 / 多分题 (**polytomous**): 有多个有序等级。例: Likert 量表「非常不同意 / 不同意 / 一般 / 同意 / 非常同意」, 编码成 0,1,2,3,4。
- 处理多级题需要更复杂的模型: PCM (偏序模型)、GPCM (广义偏序模型)。它们会给每道题估计多个阈值 (从一级跨到下一级的难度)。

1.6 单维 vs 多维

- 单维 (**unidimensional**): 整份问卷只测一个潜在特质。例: 一份纯数学测验只测「数学能力」。
- 多维 (**multidimensional**): 一份问卷同时测多个特质。例: 一份综合测验里, 前 20 题测「语文」、后 20 题测「数学」; 或一份心理量表有「焦虑」「抑郁」两个分量表。
- 在多维情形, 你需要告诉软件「每道题属于哪个维度」——这个对应关系写在一个叫 Q 矩阵的表里 (第 3.9 节详解)。
- 本手册和 IRTC 主要处理简单结构 (题间多维): 每道题只属于一个维度 (不跨维)。这是最常见的情形。

1.7 IRTC 能做和不能做什么

能做:

- 单维 / 多维的 Rasch(1PL)、PCM、RSM、2PL、GPCM 模型估计;
- 估计每道题的难度 (和区分度);
- 估计每个人的能力 (EAP 分数);
- 潜在回归: 用背景变量 (年龄、性别...) 解释能力;
- 多组: 比较不同人群 (每组各自的能力分布);
- 个案权重: 抽样调查里给每个人不同权重;
- 大数据: 内置流式引擎, 普通笔记本也能算百万人。

暂不直接做 (需要别的软件):

- 3PL (带猜测参数) 模型;
- 题内多维 (一道题同时载多个维度的复杂结构, 需用 `method="grid"` 且功能受限);
- 认知诊断、DIF 自动检验等高级专题 (可结合其他包)。

本章你应该掌握的: ① θ 是看不见的能力; ② IRT 通过「作答概率」把「人的能力」和「题的难度/区分度」放在同一把尺子上反推; ③ 难度 = 多难答对, 区分度 = 分辨高低的本事; ④ 二分题编码 0/1, 多级题编码 0,1,2,...; ⑤ 多维要用 Q 矩阵指明每题属于哪个维度。

准备你的电脑（零编程基础）

这一章手把手带你装好软件、并第一次「跑通」一段命令。不需要任何基础。

2.1 R 和 RStudio 是什么？为什么要两个？

- **R**：真正干活的统计软件（免费、开源）。你可以把它想成一个「极其强大的计算器 + Excel + 统计引擎」。但 R 自带的界面很「素」，对新手不友好。
- **RStudio**：一个给 R 用的、好看好用的工作台（界面）。你实际操作的是 RStudio，它在背后调用 R。
- 类比：R 是发动机，RStudio 是带方向盘、仪表盘的整辆车。你开车，但发动机在底下干活。两个都要装，先装 R 再装 RStudio。

2.2 第一步：安装 R

1. 打开浏览器，访问 <https://cran.r-project.org>（这是 R 的官方网站，叫 CRAN）。
2. 在最上方选择你的系统：
 - Windows：点 **Download R for Windows** → **base** → **Download R-x.x.x for Windows**，下载后双击安装，一路「下一步」即可。
 - Mac：点 **Download R for macOS** → 下载对应芯片的 **.pkg**（Apple 芯片选 arm64，旧 Intel 选 Intel），双击安装。
3. 装完后你的电脑里就有了 R。但先别打开 R 自带的界面，我们用 RStudio。

2.3 第二步：安装 RStudio

1. 访问 <https://posit.co/download/rstudio-desktop/>。
2. 下载 **RStudio Desktop**（免费版），双击安装，一路默认。
3. 装完后，打开 **RStudio**（不是 R）。它会自动找到你刚装的 R。

2.4 认识 RStudio 的界面（四块区域）

第一次打开 RStudio，你会看到屏幕被分成几块。对新手最重要的是左下角的「控制台 (Console)」：

+-----+		
脚本编辑器	环境 / 历史	
(你写命令的地方)	(看有哪些数据/变量)	
+-----+		
控制台 Console	文件/图形/包/帮助	
(命令在这里运行、	(装包、看图、看帮助)	
结果在这里显示)		
+-----+		

- **控制台 (Console)**：你输入一行命令、按回车，它立刻执行并显示结果。最简单的用法就在这里。
- **脚本编辑器 (Script)**：当你要写很多行、想保存下来反复用时，写在这里（菜单 File → New File → R Script 新建）。在脚本里把光标放到某一行，按 **Ctrl+Enter**（Mac 是 **Cmd+Enter**）就能把那行送到控制台运行。
- **环境 (Environment)**：显示你现在内存里有哪些数据、变量。
- **右下角**：装包、看图、看帮助文档的地方。

小贴士：命令前面带 **#** 的是注释，R 会忽略它，是给人看的说明。本手册代码里的中文注释都是这个作用。

2.5 第一次运行命令：把控制台当计算器

在控制台里输入下面这行，按回车：

```
1 + 1
```

你会看到：

```
[1] 2
```

[1] 是「这是第 1 个结果」的意思，**2** 是答案。恭喜，你已经运行了第一条 R 命令。

再试一个——把一个数存起来（叫「赋值」），用 **<-**（小于号加减号，像个箭头）：

```
x <- 5      # 把 5 存进一个叫 x 的盒子
x * 2       # 取出 x 乘以 2
```

输出 **[1] 10**。这里 **x** 叫变量，**<-** 叫赋值（把右边的东西装进左边的盒子）。这是 R 最核心的操作，请务必理解。

2.6 第三步：安装 IRTC 包

IRTC 是一个「插件（包）」，要先装进 R。IRTC 已经在 CRAN（R 的官方软件仓库）上架，所以最简单的办法就是一条命令。

推荐方式：从 CRAN 安装（在 RStudio 控制台里运行）

```
install.packages("IRTC")
```

按提示选一个镜像（选离你近的即可），等它跑完，看到最后一行类似 `* DONE (IRTC)` 或没有红色报错，就装好了。

Windows 和 Mac 用户注意：CRAN 会直接给你编译好的二进制包，装的时候不需要任何编译工具，几十秒就装完。如果 R 问你「是否从源码安装需要编译的包？（Do you want to install from sources...）」，回答 **no** 即可。

只有下面两种情况才需要编译工具（Linux 用户、或坚持装源码包）：

- **Windows：**访问 <https://cran.r-project.org/bin/windows/Rtools/>，下载与你 R 版本匹配的 Rtools，安装。
- **Mac：**打开「终端（Terminal）」应用，输入 `xcode-select --install` 回车，按提示安装命令行工具。
- **Linux：**安装 `r-base-dev`（如 `sudo apt-get install r-base-dev`）。

备选方式：本地源码包安装。如果你所在的网络访问不了 CRAN，或者别人直接把源码包文件 `IRTC_1.1.2.tar.gz` 发给了你：

```
# 把引号里换成你电脑上 IRTC_1.1.2.tar.gz 的真实路径
install.packages("C:/Users/你的名字/Downloads/IRTC_1.1.2.tar.gz",
                 repos = NULL, type = "source")
```

这种方式需要先按上面的说明装好编译工具。

装完检查一下版本（确认与本手册一致）：

```
packageVersion("IRTC") # 应显示 '1.1.2'
```

可选功能包：读 Excel 需要 `readxl`，读 SPSS/Stata/SAS 需要 `haven`，导出 Excel 需要 `openxlsx`，生成 Word 报告需要 `officer`。不用提前装——用到时 IRTC 会提示你一行命令安装。

「路径」是什么？就是文件在电脑里的「地址」。比如下载到「下载」文件夹，Windows 路径常是 `C:/Users/你的名字/Downloads/IRTC_1.1.2.tar.gz`，Mac 常是 `/Users/你的名字/Downloads/IRTC_1.1.2.tar.gz`。注意 R 里路径用正斜杠 `/`，不要用反斜杠 `\`。

找路径的笨办法：在「文件资源管理器/访达」里找到那个文件，右键看属性/简介里的位置；或把文件拖进 RStudio 控制台，它会自动显示路径。

IRTC 还依赖另外两个包（Rcpp、RcppArmadillo）。从 CRAN 安装时它们会自动装好；如果本地装源码包时提示缺少，先装它们：

```
install.packages(c("Rcpp", "RcppArmadillo"))
```

2.7 第四步：加载包

安装是「装到电脑里」（只需一次）；每次新开 RStudio 用 IRTC 前，都要先「加载」它（告诉 R 这次要用它）：

```
library(IRTC)
```

加载成功时，屏幕上会打印一行版本横幅：

```
* IRTC 1.1.2 (2026-08-27)
```

看到它就说明装的是本手册对应的正式发布版。如果报「there is no package called ‘IRTC’」，说明上一步安装没成功，回到 2.6。

2.8 跑通第一个完整例子（用软件自带数据）

IRTC 自带了练习数据，不用你准备，直接能跑。在控制台逐行运行：

```
library(IRTC)           # 加载包
data(data.sim.rasch)     # 自带数据：2000 人 × 40 题，0/1 作答
mod <- irtc(data.sim.rasch, model = "1PL") # 一行完成：预检 + 估计 + 质量评级
plain_summary(mod)       # 白话摘要：先结论，后细节
```

如果屏幕先滚出迭代信息、最后打印出「总体结论 / 分析概况 / 题目质量 / 能力分布 / 下一步」五段摘要，你已经完整跑通了一次 IRT 分析！想看统计人员风格的完整技术输出，再运行 `summary(mod)`。

本章你应该掌握的：① 先装 R 再装 RStudio，实际用 RStudio；② 命令在「控制台」里运行；③ `<-` 是赋值（装进盒子），`#` 是注释；④ `install.packages("IRTC")` 从 CRAN 装包（一次，Windows/Mac 无需编译工具），`library(IRTC)` 加载包（每次）；⑤ 你已经用 `irtc()` 一行命令跑通了自带数据。

把数据交给 IRTC（大部分整理是自动的）

在很多 IRT 软件里，「数据整理」是报错的最大来源。IRTC 把常见的整理工作自动化了：你把文件交给 `irtc()` 或 `irtc_read()`，它自动读取、清洗并记录日志。本章告诉你：它能自动做什么、你仍需要注意什么。

3.1 你的文件长什么样就行

IRTC 直接读取以下格式，不需要先在 Excel 里另存转换：

来源	扩展名	说明
Excel	.xlsx / .xls	需安装 readxl 包（首次使用时会提示一键安装）
文本表格	.csv / .tsv / .txt / .dat	分隔符（逗号/制表符/分号/竖线）和编码（UTF-8/GBK）自动识别
SPSS	.sav / .por	需安装 haven 包
Stata / SAS	.dta / .sas7bdat / .xpt	需安装 haven 包
R 对象	data.frame / matrix / tibble	直接传入

对文件内容的要求只有一条：一行表头（题目名）+ 每行一个人、每列一道题。学号/姓名列可以留在表里——IRTC 会自动识别并分离（见 3.2）。

```
d <- irtc_read("作答数据.xlsx") # 读取 + 自动清洗，打印清洗日志
d$resp # 清洗后的作答数据
d$pid # 自动分离出来的学号/编号
d$log # 每一步清洗的日志（中英双语，可追溯）
```

3.2 IRTC 自动帮你做的清洗

`irtc_read()`（以及一站式的 `irtc()`）自动完成，且每一步都记入日志：

1. 分离 ID 列：识别常见列名 (id、pid、学号、姓名、编号、准考证号等)，把它从作答里拿出来存到 `$pid`；也可用 `id=" 列名"` 手工指定。
2. 删除全空的行和列。
3. 文本转数字：把 "1"、"2" 这类文本数字转成数值；""、"N/A"、"缺失" 等转成缺失。
4. 缺失码转 NA：-9、-99、99、999 等常见缺失码自动转缺失。有范围保护：正数缺失码只在明显超出该题作答范围时才转换 (0~100 计分里合法的 99 分不会被误伤)；可用 `missing_codes=` 自定义。
5. 类别重编码：多级题自动重编码为从 0 开始的连续整数 (1~5 的 Likert 自动变 0~4；有空档的类别自动并拢)，逐题记日志。

3.3 多级计分 (Likert 等) ——现在是自动的

IRT 模型内部把类别当作 0,1,2,... 处理，所以多级题必须从 0 连续编号。这一步自动完成 (见 3.2 第 5 条)，你用 1~5 原始编码直接交给 `irtc()` 也没问题。

如果你想自己控制，仍可手工处理后再传入，或用 `irtc_read(..., recode=FALSE)` 关闭自动重编码：

```
dat <- dat - 1      # 手工把 1~5 改成 0~4 (传统做法，仍然有效)
```

没人选到的类别怎么办？多级题里偶尔有某个分数没有任何人得到 (比如 5 点量表里没人选某一档，或声明满分 3 分却没人拿到 3)。这类「空类别」会让标准估计不稳。`irtc()` 用 `rare_categories=` 处理，两种方案：

- "collapse" (默认)：把空类别与相邻类别并拢，并在题目输出的 `categories_unobserved/` `categories_collapsed` 列记录未观测到的类别与映射关系——常规分析用这个；
- "prior"：保留原有类别结构，对受影响的阈值施加强先验 (在 2PL/GPCM 下则固定在先验均值) 使其稳定——当你需要保持类别编号不变、跨年可比时用这个。

```
mod <- irtc("数据.xlsx", model = "PCM", rare_categories = "prior")
```

完全没人作答、或零方差的题目也不会被悄悄丢弃：它们在 `irtc_results()` 里保留一行，`status` 标为 `dropped_no_response` 或 `declared_not_estimated`，让结果表的题目数始终与你的题库对得上。

3.4 缺失值——自动转换，但要认识 NA

R 里缺失值写作 **NA** (大写，不加引号)。`irtc_read()` 已把常见缺失码自动转成 NA；若你的项目用了特殊缺失码 (比如 88)，显式声明：

```
d <- irtc_read("数据.csv", missing_codes = c(-9, 88, 99))
```

手工替换的传统做法也仍然有效：`dat[dat == -9] <- NA`。

3.5 反向计分题——这一步仍需你人工处理！

反向题（如满意度量表里的「我经常感到沮丧」）软件无法自动识别，必须在分析前人工反转，让所有题的高分指向同一方向：

```
# 5 点量表 (0~4) 反转公式: 新值 = 4 - 旧值; 假设第 3、7 题是反向题
dat[, c(3, 7)] <- 4 - dat[, c(3, 7)]
```

忘了反转的信号：题目质量表里该题「区分度为负——请排查！」（IRTC 会自动把负区分度的题标为「建议修改」并提示排查原因）。

3.6 原始 A/B/C/D 作答——用答案键自动计分

如果你的文件里存的是选项字母（A/B/C/D）而不是 0/1，不用自己改，给一个答案键即可：

```
mod <- irtc(" 原始作答.xlsx", model = "2PL",
            key = c(Q1 = "A", Q2 = "C", Q3 = "B")) # 每题的正确答案
```

答案键除了写成上面这样的向量，也可以是一份文件（.xlsx/.csv 等任意支持的格式），题目多时更好维护：文件里放一列题目名（列名 `item/题目`）和一列正确答案（列名 `answer/答案`）即可。

```
mod <- irtc(" 原始作答.xlsx", model = "2PL", key = " 答案键.xlsx")
```

部分计分（全对 = 2 / 部分对 = 1 / 其余 = 0）：在答案键文件里再加一列 `partial_answer/部分正确答案`，列出「算部分正确」的选项（一题多个用 |、;、, 或、分隔）。这样题目自动按多级（GPCM/PCM）计分：

```
# 答案键.xlsx 三列: item / answer / partial_answer
#   Q1      A      B
#   Q2      C      B|D
mod <- irtc(" 原始作答.xlsx", model = "GPCM", key = " 答案键.xlsx")
```

大小写、空格、全角字母（A）都会自动归一化。若某题有三级以上的自定义分值，用 `rules=` 传一张「题目-选项-分值」表（同样可以是文件），详见第 14 章 `irtc_score()`。若你在 Q 矩阵里把某题声明为分部计分、但答案键只给了对/错（或反之），IRTC 会给出一致性提醒（W423/W424）。

答案键按你自己数据文件里的编码书写

如果你的选项存的是数字（1/2/3/4 而不是 A/B/C/D），答案键直接写你在文件里看到的那个数字：

```
# 数据文件里 C3 题的选项编码是 1..5, 正确答案是第 2 个选项
mod <- irtc("原始作答.sav", model = "2PL", key = c(C3 = 2))
```

这一点在 1.1.2 之前是错的。读入数据时 IRTC 会把「1,2,3,4,5」这样的选项重新编号为「0,1,2,3,4」（估计需要从 0 开始的连续分数，日志码 I124）；1.1.1 及更早版本的答案键是拿去和重编号之后的值比对的，于是 `key = c(C3 = 2)` 实际判定的是原始编码 3 那个选项。没有任何警告，算出来的 0/1 矩阵看上去完全正常，通过率也在合理区间。

1.1.2 起，IRTC 会记住每题的原始选项编码，并把答案键、部分正确答案和 `rules` 计分表自动换算回去。你只需要按数据文件自身的编码写答案键——那也是你唯一能看到的编码。

还有一道安全网：如果某个答案在该题的实际作答里从未出现过（写错了，或者那个选项根本没人选），会给出 W205 警告，而不是让所有人都判错：

└─ [W205] 以下答案在该题的实际作答中从未出现，因此没有任何人会被判为答对：C3=9。

存档分析怎么排查：`irtc_audit_scoring()`

上面这个缺陷看结果是看不出来的。所以 1.1.2 提供了一个直接判定的函数，不用重跑：

```
irtc_audit_scoring(readRDS("我的模型.rds"))
```

它读得懂你手边任何一种存档：估计好的模型、`irtc_read()/irtc_score()` 的数据对象、`irtc_results()` 的结果列表，或者只剩下的清洗日志（导出的 Excel 工作簿和报告里都带着它）。结论有五种：

status	含义
not_scored	没用答案键，数据本来就是计好分的——不可能受影响
not_affected	用了答案键，但相关题目没发生重编号
ok_fixed	由 1.1.2 及以后的版本产出
affected	确认受影响， <code>\$affected_items</code> 列出需要重跑的题目
affected_if_before_1.1.2	条件命中但产出版本没有记录（1.1.2 之前不打版本戳），需要先确认版本

两类分析天然不受影响：① 直接喂入已经计好分的 0/1 数据（不传 `key=`）；② 传了 `key=`，但题目选项本来就从 0 开始连续编号。

3.7 分析前体检: `irtc_check_data()`

不放心数据? 先体检。它会检查: 题目/样本是否够、有没有文字列、负数或非整数取值、零方差题、极端缺失、稀疏类别、全空的人、重复学号——每个问题都给出处理建议:

```
chk <- irtc_check_data(irtc_read("数据.csv"))
chk$ok          # TRUE = 可以直接估计
chk$issues      # 问题清单: 错误码、严重程度、位置、建议
```

`irtc()` 内部会自动做这个体检: 发现「错误级」问题会停下来把清单告诉你 (错误码 E407); 「警告级」问题 (如零方差题) 会自动处理并告知。

传统的手工体检命令 (`dim`、`str`、`range`、`table`) 仍然是好习惯, 见第 14 章速查。

3.8 常见数据问题: 现在谁来管?

现象	V0.1 时代要你自己做	V1.0 谁来管
Likert 用了 1~5	<code>dat <- dat - 1</code>	<code>irtc_read()</code> 自动重编码并记日志
缺失填了 -9/99	手工替换成 NA	自动转换 (带范围保护)
混进学号/姓名列	手工删除	自动识别分离到 <code>\$pid</code>
文本数字"1"	手工转数值	自动转换
A/B/C/D 原始作答	手工改 0/1	<code>key=</code> 答案键自动计分
反向题没反转	自己记得	仍需人工 (质量表会用负区分度提醒你)
全空行列 / 零方差题	手工排查	自动删除/剔除并告警
数据到底行不行	逐项目视检查	<code>irtc_check_data()</code> 一键体检

3.9 多维分析必备: Q 矩阵怎么写?

如果你的问卷测多个维度 (比如「语文」和「数学」两个分量表), 你要告诉软件每道题属于哪个维度。这通过一张 Q 矩阵实现:

- Q 矩阵的行 = 题, 列 = 维度;
- 格子是 0 或 1: 第 j 题属于第 d 维就在 `Q[j,d]` 放 1, 否则放 0;
- 简单结构下, 每行只有一个 1 (每题只属于一个维度)。

最省事的写法是把 Q 矩阵也做成一张表: 第一列是题目名 (列名 `item/题目`, 要与作答数据的题目名一致), 其余每一列是一个维度, 列名就是维度名。用小写的 `q=` 交给 `irtc()`:


```
# Q 矩阵.xlsx: item / 语文 / 数学 (前 4 题测语文, 后 4 题测数学)
# Q1      1      0
# ...
# Q8      0      1
mod <- irtc("数据.xlsx", model = "2PL", q = "Q 矩阵.xlsx")
```

q= 也可以直接传一个 data.frame, 或一个带题目行名的数值矩阵。用它有三个好处:

- 维度名会自动带到所有个人结果里——能力值列变成 eap_ 语文/eap_ 数学、标准误变成 se_ 语文/se_ 数学, 不再是无意义的 dim1/dim2;
- Q 矩阵与作答数据的题目会自动对齐: 只在一边出现的题会给出提醒 (默认 on_mismatch="warn" 保留共同题; on_mismatch="error" 则报错停下);
- Q 矩阵表里可以再加一列 分部计分/满分 (列名 partial/max_score 等), 一并声明哪些题多级计分、满分几分。

单独读取/检查 Q 矩阵: irtc_read_q("Q 矩阵.xlsx") 读入并规范化, irtc_align_q(数据, q) 单独做对齐检查。

专家透传: 若你直接调用 irtc.mml() 系列, 或想把已排好的数值载荷矩阵原样交给估计引擎, 用大写的 Q= (不做题名对齐、也不生成维度名): Q <- cbind(c(1,1,1,1,0,0,0,0), c(0,0,0,0,1,1,1,1))。

单维分析不需要 Q 矩阵 (默认单维)。

3.10 背景变量、分组、权重怎么单独存?

「年龄、性别、地区、抽样权重」这些不是作答, 不能算进题目里。学号/姓名列 IRTC 会自动分离; 其余背景变量单独取出, 通过参数传入 (全部透传给估计引擎):

```
raw <- irtc_read("原始表.xlsx", id = "学号")$resp
gender <- raw[["性别"]] # 分组变量
resp <- raw[, setdiff(names(raw), "性别")] # 剩下的才是作答

mod <- irtc(resp, model = "2PL", group = gender) # 多组比较
mod <- irtc(resp, model = "1PL", Y = cbind(1, age)) # 潜在回归 (见 4.5 的 Y)
```

抽样权重如果就在你的数据文件里, 不用手工分离: irtc()/irtc_read() 会自动识别常见的权重列名 (weight、wt、pweight、权重、样本权重、抽样权重等), 把它分离出来并在估计时作为 pweights 使用; 权重必须为正数, 缺失的自动置 1 并告警, 加权样本量会打印出来。也可以用 weights= 显式指定是哪一列:

```
mod <- irtc("含权重的数据.xlsx", model = "1PL") # 自动识别权重列
mod <- irtc("数据.xlsx", model = "1PL", weights = "w") # 显式指定权重列
```

单字母列名 `w` 不会被自动当成权重（避免误伤名叫 `w` 的题目列），这种情况请用 `weights="w"` 显式指定。若你已经把权重取成一个向量，也可以照旧用 `pweights=` 透传。

1.1.2 起，权重会贯穿整条结果链路：不只进入 IRT 估计，经典难度（通过率）、题总相关、Cronbach's alpha、Infit/Outfit、由它们判定的质量评级，以及个人结果里的百分位和 T 分数，全部按加权口径计算。1.1.1 及更早版本只有 IRT 参数是加权的，同一张结果表里紧挨着的经典统计量却是未加权的——在真实调查中这个差别可以达到 7~8 个百分点，足以改变一道题的难度标签。详见 4.1。

本章你应该掌握的：① 六类文件格式直接交给 `irtc()/irtc_read()`，清洗自动完成且有日志；② 多级重编码、缺失码、ID 列、文本数字都不用你管；没人得到的类别用 `rare_categories=` 处理；③ 反向题必须自己反转（质量表的负区分度是安全网）；④ A/B/C/D 用 `key=` 计分，答案键可以是文件、加 `partial_answer` 列即分部计分；⑤ 不放心先 `irtc_check_data()` 体检；⑥ 多维用 `q=` 传 Q 矩阵表，维度名自动带进结果；⑦ 权重用 `weights=`（或自动识别），其余背景/分组单独传参。

核心用法：从一行命令到完整交付

本章前半（4.1–4.4）是主 workflow——绝大多数分析用这几个函数就够了；后半（4.5–4.9）是专家层估计函数的逐参数详解，需要精细控制时再读。全部函数的速查表在第 14 章。

4.1 主入口：irtc() 一行命令

```
mod <- irtc("作答数据.xlsx", model = "1PL")
```

这一行自动完成：读取 → 清洗 → （计分）→ 体检 → 估计 → 质量评级。参数：

参数	这是什么、放什么	例子
data	文件路径（六类格式，见 3.1）、数据框或 <code>irtc_read()</code> 结果。必填。	"数据.xlsx"
model	模型。必填："1PL"（同 "Rasch"）、"2PL"、"PCM"、"PCM2"、"RSM"、"GPCM"，大小写随意。	model = "GPCM"
key / rules	原始 A/B/C/D 作答的答案键 / 多级计分规则表（见 3.6）；可以是向量/表，也可以是文件；答案键加 <code>partial_answer</code> 列即分部计分。	key = "答案键.xlsx"
q	Q 矩阵（题目 × 维度）表或文件，声明多维结构；维度名自动带进结果（见 3.9）。	q = "Q 矩阵.xlsx"

参数	这是什么、放什么	例子
<code>on_mismatch</code>	Q 矩阵与数据题目不一致时： "warn"（默认，保留共同题） 或 "error"（停止）。	"error"
<code>weights</code>	抽样权重的列名/列号；不填 则自动识别常见权重列（见 3.10）。	<code>weights = "w"</code>
<code>rare_categories</code>	没人得到的类别怎么处理： "collapse"（默认，并拢） 或 "prior"（保留，见 3.3）。	"prior"
<code>id</code>	手工指定 ID 列（默认自动识 别）。	<code>id = " 学号"</code>
<code>missing_codes</code>	自定义缺失码（见 3.4）。	<code>c(-9, 88)</code>
<code>check</code>	体检发现错误时是否停止 （默认 <code>TRUE</code> ，附完整问题清 单）。	一般不动
<code>quality</code>	是否附带质量评级/CTT/拟 合（默认 <code>TRUE</code> ）。	一般不动
...	任何专家参数（大写载荷矩 阵 <code>Q</code> 、 <code>group</code> 、 <code>pweights</code> 、 <code>Y</code> 、 <code>control</code> 等）原样透传给 4.5/ 4.6 的估计函数。	<code>group = g</code>

怎么选 **model**? 对错题 (0/1): 想要最稳健、可比性强用 "1PL"; 想估计每题区分度用 "2PL"。多级计分/量表题: "PCM" (不估区分度) 或 "GPCM" (估区分度)。同一套 Likert 选项的整卷可用 "RSM"。

返回标准 `irtc` 模型对象，另含 `mod$usability` (清洗日志、体检结果、CTT、拟合、质量评级)。全空题和零方差题会自动剔除并告警 (W410)。

权重是贯穿的 (1.1.2 起): 你给的 `weights=` 不只送进 MML 估计，还会自动下传给经典统计、题目拟合、质量评级和个人常模列，因此一张结果表里所有数字都描述同一个人群：

输出	加权?
IRT 题目参数 (区分度、难度)、潜在方差/相关	是
经典难度 (通过率)、题总相关、Cronbach's alpha	是
Infit / Outfit 及其 t 统计量、四级质量评级	是

输出

加权?

个人结果的 `percentile` (百分位)、`t_score` (T 分数) 是

个人能力值 `eap` 与标准误 `se`

否——这是个体量，不随权重变

不使用权重的分析完全不受影响：权重全部相同时，加权公式与未加权公式逐位相同（并列情形也一样）。

4.2 白话结果：`plain_summary()` 与质量评级

```
plain_summary(mod)           # 五段式白话摘要：总体结论 → 概况 → 题目质量 → 能力分布 → 下一步
mod$usability$quality        # 每题评级：好 / 可用 / 需检查 / 建议修改（附原因和建议）
plot(mod, type = "wright")   # 题目难度-人群能力对照图；还有 "ability" / "quality" / "icc"
```

评级规则（阈值可调，见 `irtc_quality_thresholds()`）：得分率 <10% 或 >95% 标准度问题；区分度 <0.25 标偏低、<0.15 很低、负数直接判「建议修改」（多为答案键错误）；拟合 `outfit/infit` 超出 0.7–1.3 标偏离。

4.3 交付三件套：Excel、报告、图

```
irtc_excel(mod, dir = "结果")
```

一次生成三个独立 Excel（需 `openxlsx`，首次使用会提示安装）：

1. 题目质量表 `*_item_quality.xlsx`：红黄绿配色评级 + 白话原因和处理建议 + 逐列说明页——不懂统计的同事也能直接判断题目好坏；
2. 题目参数表 `*_item_parameters.xlsx`：列结构冻结（schema v1.0），跨年度锚题只要 `item_id` 一致就能直接合并做链接等值；
3. 能力值表 `*_person_ability.xlsx`：行顺序与输入数据一致（学号、作答题数、原始分、能力值 EAP、标准误、百分位、T 分数），可整体粘贴进总样本表。

```
irtc_report(mod, "报告.docx", audience = "decision") # 给决策者：1~2 页结论摘要
irtc_report(mod, "报告.html", audience = "survey")   # 给调查人员：白话全文（html 零依赖）
irtc_report(mod, "报告.html", audience = "stat")     # 给统计人员：完整技术版 + ICC 曲线
```

`survey/stat` 报告还含两节：模型诊断（是否收敛、AIC/BIC、EAP 信度分档、拟合解读）和数据处理透明度（是否加权及权重分布、Q 矩阵对齐、类别并拢、被剔除的题、计分方式、清洗日志）——把「数据到底做了哪些处理」一次交代清楚。输出文件的上级目录若不存在会自动创建。

4.4 语言切换与 AI/自动化接口

```
options(irtc.lang = "en")      # 所有输出切换为英文（默认 "zh" 中文）

res <- irtc_results(mod)        # 结构冻结的机器可读结果（模型信息/题目/个人/日志/体检）
irtc_json(mod, "results.json") # 同样内容导出 JSON（需 jsonlite）
```

所有报错带 `$code`（错误码）、`$reason`（原因）、`$fix`（修复建议）三个字段（见第8章），自动化脚本可直接 `tryCatch` 处理。包内 `inst/llms.txt` 是给 AI 助手准备的 API 速览。

以下 4.5–4.9 为专家层。`irtc()` 的 `...` 会把这里的所有参数原样透传，所以你可以在一行命令里使用任何专家功能；也可以直接调用这两个估计函数（它们只接受整理好的数值矩阵，不做自动清洗）。

4.5 `irtc.mml()`：固定斜率模型（Rasch / PCM / RSM）

什么时候用它？当你想用不估计区分度的模型——即假设所有题区分力相同。这类模型最经典、最稳健，包括：

- **1PL / Rasch**：二分题，只估难度；
- **PCM（偏序模型）**：多级题，估每个阈值；
- **RSM（评定量表模型）**：多级题，且各题阈值结构相同（适合同一套 Likert 选项的整份问卷）。

最简单的调用：

```
mod <- irtc.mml(resp = dat)          # 默认 1PL/Rasch
mod <- irtc.mml(resp = dat, irtmodel = "PCM") # 多级题用 PCM
```

全部常用参数逐个解释：

参数	这是什么、放什么	例子
<code>resp</code>	你的作答矩阵（人 × 题）。必填。	<code>resp = dat</code>
<code>irtmodel</code>	选哪种模型："1PL"（二分 Rasch）、"PCM"（多级偏序）、"PCM2"（PCM 的另一种参数化）、"RSM"（评定量表）。	<code>irtmodel = "PCM"</code>
<code>Q</code>	Q 矩阵（题 × 维），指明每题属于哪维。单维可不填。	<code>Q = Qmat</code>

参数	这是什么、放什么	例子
<code>ndim</code>	维度数。不写 <code>Q</code> 时用它说明有几维（默认 1）。	<code>ndim = 2</code>
<code>Y</code>	潜在回归的背景变量矩阵（人 × 变量）。想「用年龄/性别解释能力」时填。一般第一列放全 1（截距）。	<code>Y = cbind(1, age)</code>
<code>group</code>	分组向量（长度 = 人数），做多组比较时填，如性别。	<code>group = gender</code>
<code>pweights</code>	个案权重（长度 = 人数，非负），抽样调查填。	<code>pweights = w</code>
<code>xsi.fixed</code>	把某些题目参数固定成已知值（高级用法，做等值/锚题时用）。	一般留空
<code>variance.fixed</code>	固定（协）方差的某些元素（高级）。	一般留空
<code>est.variance</code>	是否估计潜在能力的方差（默认 <code>TRUE</code> ）。	<code>est.variance = TRUE</code>
<code>pid</code>	每个人的 ID（不影响估计，只是给结果贴标签）。	<code>pid = id_vec</code>
<code>verbose</code>	是否在屏幕打印迭代进度（默认 <code>TRUE</code> ）。嫌烦设 <code>FALSE</code> 。	<code>verbose = FALSE</code>
<code>control</code>	控制参数列表，微调算法（见 4.8）。	<code>control = list(maxiter=500)</code>

完整例子：

```
# 多维 PCM：用 Q 矩阵，关闭进度打印
m <- irtc.mml(resp = dat, irtmodel = "PCM", Q = Qmat, verbose = FALSE)

# 潜在回归：用「是否城市 (city=0/1)」预测能力
m <- irtc.mml(resp = dat, Y = cbind(1, city))

# 抽样权重
m <- irtc.mml(resp = dat, pweights = w)
```

返回什么？一个 `irtc` 类的「结果对象」，你把它存进 `m`，再用第 5 章的方法去读。里面有 `m$xsi`（题目参数）、`m$person`（个人能力）、`m$variance`（方差）、`m$ic`（信息准则）等。

4.6 `irtc.mml.2pl()`: 含区分度模型 (2PL / GPCM)

什么时候用它？当你想估计每道题的区分度（不假设所有题一样），或处理大数据。包括：

- **2PL**：二分题，估难度和区分度；
- **GPCM**（广义偏序）：多级题，估区分度和各阈值；
- 变体：`GPCM.design`、`2PL.groups`、`GPCM.groups`（高级，见帮助页）。

这是大数据「流式引擎」的入口。

最简单的调用：

```
m <- irtc.mml.2pl(resp = dat, irtmodel = "2PL")           # 二分 2PL
m <- irtc.mml.2pl(resp = dat, irtmodel = "GPCM", Q = Qmat) # 多级 GPCM
```

在 4.5 参数之外，新增/重点参数：

参数	这是什么、放什么	例子
<code>irtmodel</code>	"2PL"、"GPCM"、 "GPCM.design"、 "2PL.groups"、 "GPCM.groups"。	<code>irtmodel = "GPCM"</code>
<code>method</code>	选哪个计算引擎："auto" (自动，推荐)、“grid”（标准 全网格）、“streaming”（大 数据流式）。详见 4.7。	<code>method = "auto"</code>
<code>est.slopegroups</code>	让某些题共享同一个区分度 (把题分组)。	高级，一般留空
<code>E</code>	区分度的设计矩阵 (<code>GPCM.design</code> 用)。	高级
<code>control</code>	控制参数，流式与加速选项 都在这里（见 4.8）。	<code>control = list(fast=TRUE)</code>

完整例子：

```
# 自动选引擎（推荐）
m <- irtc.mml.2pl(resp = dat, irtmodel = "GPCM", Q = Qmat)
m$routing      # 看它选了哪个引擎、为什么

# 多组：比较男女，每组各自的能力分布
m <- irtc.mml.2pl(resp = dat, irtmodel = "2PL", Q = Qmat,
```



```

                                group = gender, method = "streaming")
m$variance      # 多组时是「列表」，每组一个协方差
m$gmean         # 各组的平均能力（参照组固定为 0）

```

4.7 method: 三个引擎到底怎么选？

`irtc.mml.2pl` 有三种「计算引擎」，结果目标一致，只是速度和适用范围不同：

- **"grid"**（网格引擎）：最标准、最稳的全网格数值积分。任何受支持的模型都能用，小数据首选。缺点：维度高、数据大时会慢、占内存。
- **"streaming"**（流式引擎）：为大型、题间多维（简单结构）、**2PL/GPCM** 设计的高性能引擎，内存占用很小（百万人也不爆内存），支持协变量/组/权重。遇到它不支持的模型（如题内多维）会报错提示你改用 `grid`。
- **"auto"**（自动，默认）：软件自己预测哪个引擎更快（根据人数、题数、维度、节点数、协变量模式数），然后自动选。新手就用默认的 **auto**，不用纠结。结果里 `m$routing` 会告诉你它选了谁、为什么。

一句话：不确定就用 `method="auto"`（默认）。只有当你明确知道要强制某引擎时才手动指定。

4.8 control: 控制参数全清单（逐项解释）

`control` 是一个列表，用来微调算法。写法是 `control = list(参数 1 = 值 1, 参数 2 = 值 2)`。新手大多数情况留空即可，下面解释每一项，按「你最可能用到」排序。

A. 基本收敛/积分控制

项	含义	默认	什么时候改
<code>maxiter</code>	最大迭代次数（算法最多算多少轮）	1000	提示「未收敛」时调大
<code>conv</code>	参数收敛阈值（变化小于它就停）	1e-4	一般不动
<code>convD</code>	偏差收敛阈值	1e-3	一般不动
<code>nodes</code>	每个维度的积分节点（一个数值向量）	<code>seq(-6, 6, len=21)</code>	高维想更快可减少；想更精可增多
<code>Msteps</code>	每轮内部小迭代数	4	一般不动
<code>acceleration</code>	EM 加速: "none"/ "squarem"/ "Ramsay"/"Yu"	"none"	收敛慢时用 "squarem" 提速

项	含义	默认	什么时候改
<code>ridge</code>	协方差「岭」正则 (数值稳定用的小量)	0	协方差报奇异时设个小值如 <code>1e-4</code>
<code>verbose/progress</code>	是否打印进度	TRUE	嫌烦设 FALSE
<code>n_threads</code>	并行线程数	2 (最大 2)	可设为 1 以使用单线程；为符合 CRAN 政策不可超过 2

B. 大数据加速（流式 **fast** 模式）

项	含义	默认
<code>fast</code>	是否开启近似加速（剪掉不重要的积分节点）	FALSE（精确）
<code>mass_budget</code>	加速的「预算 ϵ 」：愿意丢掉多少先验质量换速度。注意：它是预算，不是保证误差（真正的误差由下面的核查报告给出）	<code>1e-3</code>
<code>verify</code>	加速后的误差核查方式： "stratified"（分层抽样，推荐）/"random"/FALSE（不查）	"stratified"
<code>verify_n</code>	核查用多少人	3000
<code>verify_seed</code>	核查抽样的随机种子（保证每次核查可复现）	1
<code>refine</code>	如果核查没达标，自动收紧预算重算直到达标	FALSE
<code>tol_deviance / tol_eap / tol_moment / tol_par</code>	核查时各指标的容差（偏差/能力分/矩/参数）	各有默认

C. 多组与正则（高级）

项	含义	默认
group_structure	多组协方差结构: "full" (每组独立协方差) / "mean" (共享协方差、只放开组均值)	"full"
reg	统计正则 (流式): 列表, 可 含 slope_penalty (区分度惩 罚)、slope_max (区分度上 限)、sigma_shrink/sigma_ shrink_pooled (协方差收缩)	关

加速 + 核查的标准写法 (大数据放心用):

```
m <- irtc.mml.2pl(resp = bigdat, irtmodel = "GPCM", Q = Qmat,
  control = list(fast = TRUE, mass_budget = 1e-3,
    verify = "stratified", verify_n = 3000, refine = TRUE))
m$accuracy_report$met      # TRUE = 加速带来的实测误差在容差内, 结果可放心用
m$accuracy_report          # 看详细的误差报告 (中位数/95% 分位/最大值)
```

4.9 查看结果的 4 个方法 (S3 方法)

把结果对象 (比如 m) 传给这些函数:

方法	作用	怎么用
summary(m)	打印可读的结果摘要 (题目 参数、方差、信息准则)。最 常用。	summary(m)
print(m) 或直接 m	简要打印对象。	m
logLik(m)	取对数似然 (衡量模型拟合 好坏的数), 可配合 AIC(m)、 BIC(m)。	AIC(m)
anova(m1, m2)	比较两个嵌套模型 (似然比 检验), 看复杂模型是否显著 更好。	anova(m1, m2)

本章你应该掌握的: ① irtc(文件, model=) 一行完成主工作流, ... 透传全部专家参数;
② plain_summary 读结论、\$usability\$quality 看每题评级; ③ irtc_excel 出三个交付表、
irtc_report 出三种受众报告; ④ options(irtc.lang=) 切换双语, irtc_results/irtc_json 给

自动化用；⑤ 精细控制时用 4.5-4.9 的专家参数（`method="auto"`、`control` 里 `fast+verify` 是大数据关键）。

读懂结果（每个输出讲透）

读结果有三条路径，按需选择：白话路径（给自己和同事）、表格路径（给交付）、技术路径（给论文和审稿人）。本章先带你走前两条，再逐块讲透技术输出。

5.1 三条读结果的路径

路径一：白话摘要（30 秒了解全局）

```
plain_summary(mod)
```

五段输出——总体结论（几人几题、信度多高、几道题能用）→ 分析概况 → 题目质量分布 → 能力分布 → 下一步建议。信度的解读已翻译成文字（非常好/良好/尚可/偏低）。

路径二：质量评级与交付表（判断每道题）

```
mod$usability$quality      # 每题：得分率、区分度、拟合、四级评级、原因、建议
irtc_excel(mod, dir = "结果") # 生成配色的 Excel 版，发给不用 R 的同事
```

路径三：完整技术输出（下面各节逐块讲透）

```
summary(mod)
```

典型的 `summary` 输出包含：① 基本信息（样本量、题数、维度、迭代）；② 信息准则（Deviance、AIC、BIC）；③ 题目参数；④ 潜在分布（均值、方差/协方差）。以下各节教你逐块读懂，并知道「什么样的数算好」。

5.2 题目参数表 `m$xsi` 怎么读？

直接看 `m$xsi`：

```
m$xsi
#           xsi      se.xsi
# I1 -1.959017 0.06465854
```

```
# I2 -1.857027 0.06311470
# ...
```

- 行名 **I1**, **I2**, ...: 第1题、第2题……
- **xsi** 列: 这道题的难度参数 (item intercept/difficulty)。
 - 在本软件参数化里, 模型是 $\text{logit}(\text{答对}) = \theta - \text{xsi}$, 所以 **xsi** 就是难度:
 - * **xsi** 越大 → 题越难 (要更高能力才答对);
 - * **xsi** 越小 (越负) → 题越容易。
 - 上例 I1 的 **xsi**=-1.96, 说明这是一道很容易的题。
 - 经验范围: 大约 -3 到 +3。超出这个范围 (如 ±5) 通常说明题极端难/极端易, 或数据有问题。
- **se.xsi** 列: 这个难度估计的标准误 (standard error), 衡量「这个估计有多准」。越小越准。如果某题 **se** 特别大 (如 >0.5), 说明这道题信息量少、估计不稳 (可能作答人太少或题太极端)。

多级题 (PCM/GPCM): 每道题会有多行/多个阈值参数 (从第0档跨到第1档、第1档跨到第2档…的难度), 命名类似 **I1_Cat1**、**I1_Cat2**。阈值应当随档次递增 (越高档越难够到); 若出现「阈值乱序」, 是「无序阈值」现象, 提示该题或某类别有问题。

导出的题目参数表 (`irtc_excel()` 的 `*_item_parameters.xlsx`, 或 `irtc_param_table()`) 会把这些阈值换算成好懂的难度标签: 三档题给 **b_partial** (得部分分的难度) 和 **b_full** (得满分的难度); 四档及以上给 **b_step1..b_stepK**。若某题有没人得到的类别, 还会附 **categories_unobserved/categories_collapsed** 列说明 (见 3.3)。

5.3 区分度怎么看? (仅 2PL/GPCM)

`irtc.mml.2pl` 的结果里, 区分度存在斜率数组 **m\$b** 中, 或在 **summary** 的题目参数部分一并打印。读法:

- 区分度 **a** 越大, 这道题分辨高低能力的本事越强。
- 经验范围: 大约 0.5 ~ 2.5。
 - **a** < 0.3: 区分力很弱, 几乎是「坏题」, 考虑删除或修订。
 - **a** 在 0.8 ~ 2: 通常是「好题」。
 - **a** 出现负数: 警报! 几乎一定是反向题没反转 (回 3.5), 或该题与其余题测的方向相反。
 - **a** 极大 (如 > 4): 可能是「数据分离」(这道题几乎完美区分), 估计不稳, 可用 `reg=list(slope_max=4)` 设上限。

5.4 个人能力分 **m\$person** 怎么用?

看 **m\$person** (每行一个人):

```
names(m$person)
# "pid" "case" "pweight" "score" "max" "EAP" "SD.EAP"
```

- **pid / case**: 人的编号。
- **score**: 原始总分（答对题数 / Likert 加总），仅供参考。
- **max**: 该人最高可能分。
- **EAP**: 这个人的能力估计（EAP = Expected A Posteriori, 后验期望）。这就是 IRT 给出的、考虑了题目难易的「科学能力分」，可以替代原始总分用于排名、分组、相关分析。
 - EAP 的量纲：默认潜在能力均值约为 0、方差由模型估计。所以 EAP 正数 = 高于平均，负数 = 低于平均，类似标准分。
- **SD.EAP**: 这个人能力估计的标准误（不确定性）。越小越准。题目越多、信息越足，SD.EAP 越小。

多维：面向交付的个人结果（`irtc_results()`\$persons、`irtc_excel()` 的能力值表）里，每个维度各有一列能力值和标准误，列名直接用你在 Q 矩阵里写的维度名——例如维度叫「语文」「数学」，列就是 `eap_ 语文/se_ 语文`、`eap_ 数学/se_ 数学`，一眼看懂是哪个维度（见 3.9）。1.1.2 起，走 streaming 引擎的多维模型也会给出标准误列（此前只有 grid 引擎有）。

pid 与 pweight (1.1.2 修正): 多维 streaming 路径此前会把你传入的 `id=` 丢掉、退化成 1,2,3... 的行号，`pweight` 也恒显示为 1。现在两者都与 grid 引擎一致：`pid` 是你自己的编号，`pweight` 是归一化到均值 1 的权重。

百分位与 T 分数是加权的（1.1.2 起）：`irtc_results()`\$persons 与能力值表里的 `percentile`、`t_score` 按加权样本计算，即「推及人口中低于该个体的比例」。能力值 `eap` 与标准误 `se` 不加权。如果你用 1.1.2 之前的版本产出过加权分析的常模分，这两列的数值会变（见附录 D）。

取出来用：

```
ability <- m$person$EAP      # 拿到每个人的能力分向量
hist(ability)                # 画能力分布直方图
# 也可以把 EAP 接回你的原始数据，做后续分析
```

5.5 潜在分布 m\$variance / m\$gmean 怎么读？

- 单维：`m$variance` 是一个数，是能力的方差（离散程度）。例 1.40 表示能力分布的方差为 1.40。
- 多维：`m$variance` 是一个协方差矩阵：对角线是各维方差，非对角线是维间协方差；换算成相关系数能看「两个维度有多相关」：

```
cov2cor(m$variance)      # 把协方差矩阵转成相关矩阵，看维度间相关
```

- 多组 (用了 `group`): `m$variance` 是一个列表, 每组一个协方差矩阵; `m$gmean` 是各组的平均能力 (参照组固定为 0, 其余组相对它)。这就是你做「人群比较」时要看的关键结果: 哪个组平均能力更高、离散更大。

5.6 模型比较: 对数似然、AIC、BIC

这些数 (在 `m$ic` 里, `summary` 也会打印) 用来比较不同模型谁更好:

- **Deviance (偏差)** = $-2 \times$ 对数似然: 越小, 拟合越好。但更复杂的模型总会拟合得更好, 所以不能只看它。
- **AIC、BIC**: 在「拟合好」和「别太复杂」之间权衡, 越小越好。
 - 比较两个模型 (比如 1PL vs 2PL) 时, **AIC/BIC** 更小的那个更受青睐。
 - **BIC** 比 **AIC** 更「惩罚复杂度」, 倾向选更简单的模型; 样本大时常看 **BIC**。
- 嵌套模型 (一个是另一个的特例, 如 Rasch 是 2PL 的特例) 可用似然比检验:

```
m1 <- irtc.mml(resp = dat)                # Rasch (简单)
m2 <- irtc.mml.2pl(resp = dat, irtmodel = "2PL") # 2PL (复杂)
anova(m1, m2)                             # 看 p 值: 显著 (<0.05) 则复杂模型 2PL 明显更好
```

```
AIC(m1); AIC(m2)    # 直接比 AIC
BIC(m1); BIC(m2)
```

1.1.2 修正: 跨引擎、跨维度的比较现在才可靠。1.1.1 及更早版本中, 多维 streaming 引擎报告的对数似然带一个与样本量成正比的系统性偏移 (等于 $n \times \text{维度数} \times \log(1/\text{节点步长})$), 因此:

- 同一维度、同一节点设定下的两个模型互比时偏移抵消, 结论仍对;
- 但单维 vs 多维、或 **grid** 与 **streaming** 混用比较时, **AIC/BIC** 会完全失效;
- 改变 `control$nodes` 会静默改变报告出来的对数似然值。

1.1.2 修正后, 两个引擎报告的 **deviance / loglike** / 各信息准则在同一尺度上, 可以直接用 `anova()`、`AIC()`、`BIC()` 跨引擎、跨维度比较。如果你以前据此得出过「几维模型更好」的结论, 请重新比较。

维度退化的提醒 (w427): 如果各维度间的潜在相关被估计到边界 1.000, 说明在这份数据上各维度分不开、模型已退化为更低维的结构。这往往是一个有价值的实质结论 (「不必分维度」), 1.1.2 会明确警告, 并提示此时的信息准则与 EAP 信度要谨慎解读。

5.7 加速后必看: `m$accuracy_report`

如果你开了 `control=list(fast=TRUE)`, 结果里会多一个 `m$accuracy_report` (误差核查报告)。这是判断「加速结果能不能信」的依据:


```

m$accuracy_report$met          # 总判定: TRUE = 实测误差在容差内, 可放心用
m$accuracy_report$eap_abs      # 能力分误差的 中位数/95% 分位/最大值
m$accuracy_report$deviance_rel # 偏差的相对误差
m$accuracy_report$nodes_kept   # 加速后保留了多少积分节点
m$accuracy_report$nodes_full    # 原本有多少节点 (对比看省了多少)

```

- **met == TRUE**: 加速带来的误差经过分层抽样实测, 确认在你设定的容差内——结果可信。
- **met == FALSE**: 误差超标。处理: 调小 **mass_budget** (如 **1e-4**), 或设 **refine=TRUE** 让它自动收紧重算。
- 如果看到提示「**kept > 70% of nodes**」, 说明加速没省多少, 不如直接关掉 **fast** 用精确模式。

5.8 路由信息: m\$routing

用 **method="auto"** (或 **streaming**) 时, **m\$routing** 告诉你软件选了哪个引擎、为什么:

```

m$routing$engine    # "grid" 或 "streaming"
m$routing$reason     # 文字理由, 如 "streaming faster (pred 19.1x)"

```

这只是「幕后信息」, 不影响结果正确性, 了解即可。

5.9 信度与测量精度: 我的问卷「准」吗?

调查工作者最常被问:「你这套题靠谱吗?」在 IRT 里,「靠谱」有两层含义。

第一层: 个人能力估计有多准——看 **SD.EAP**。每个人的 **SD.EAP** (能力估计的标准误) 越小, 这个人的分数越可信。一个粗略的经验:

- **SD.EAP < 0.3**: 很准 (题目对这个能力水平提供了充足信息);
- **0.3 ~ 0.5**: 尚可;
- **> 0.5**: 偏粗 (这个人所处的能力区问题目信息不足, 可能因为题太少、或没有难度匹配他的题)。

```

mean(m$person$SD.EAP)    # 平均测量精度, 越小越好
hist(m$person$SD.EAP)     # 看哪些人估得不准 (往往是能力极高/极低者)

```

第二层: 整份问卷的「边际信度」——类似传统的信度系数。可以用「能力的真实方差 ÷ (真实方差 + 平均测量误差方差)」估算一个边际信度 (**marginal reliability**), 含义类似 Cronbach's α , 越接近 1 越好 (一般 > 0.8 算可接受, > 0.9 算优秀):

```

# 单维近似算法:
var_theta <- as.numeric(m$variance)    # 能力真实方差
err_var   <- mean(m$person$SD.EAP^2)  # 平均测量误差方差

```

```
rel <- var_theta / (var_theta + err_var)    # 边际信度
rel
```

为什么 IRT 的信度比传统 α 更有用？因为 IRT 能告诉你信度在不同能力水平上是不一样的：题目大多是中等难度时，对中等能力者测得准（信度高），对极端能力者测不准（信度低）。这正是 SD.EAP 随能力变化所反映的。

5.10 怎样判断「好题 / 坏题」并决定删改？

把第 5.2、5.3 节的指标综合起来，给每道题打分。一道好题通常满足：

指标	好题的范围	坏题的信号
区分度 a (2PL/GPCM)	0.8 ~ 2.0	< 0.3（没区分力）或 < 0（方向反了）
难度 b / xsi	-2 ~ +2，且与你的人群匹配	超出 ± 3 （太极端，几乎全对或全错）
标准误 se.xsi	越小越好	很大（如 > 0.5，估计不稳）
多级阈值	随档次递增（有序）	阈值乱序（无序阈值）

实操：一次性筛出可疑题。

```
# 以 2PL 为例，把区分度和难度并排看
a <- m$person                                # （区分度在 summary 里；此处示意流程）
diff <- m$xsi$xsi                            # 各题难度
se <- m$xsi$se.xsi                           # 各题标准误
bad_extreme <- which(abs(diff) > 3)           # 太难/太易的题
bad_unstable <- which(se > 0.5)               # 估计不稳的题
bad_extreme; bad_unstable                     # 这些题号需要人工复核
```

决策原则：

- 区分度为负 → 几乎一定是反向题没反转，先回 3.5 处理，而不是删题。
- 区分度极低 (<0.3) → 这道题没用，考虑删除或重新编制。
- 难度极端 → 如果你的人群里这道题几乎全对/全错，它提供的信息少，可删；但如果你故意要测极端人群，则保留。
- 删题后要重新估计（题目集变了，参数会变）。

5.11 把 EAP 能力分接回你的原始数据

分析的最终目的常是「给每个人一个能力分，再做后续研究」。把 EAP 接回去：

```
# 假设 raw 是含背景变量的原始数据框，行序与作答矩阵一致
raw$ability <- m$person$EAP          # 新增一列「能力分」
raw$ability_se <- m$person$SD.EAP    # 新增「能力分标准误」

# 之后就能像普通变量一样用，例如看能力与年龄的相关：
cor(raw$ability, raw$age, use = "complete.obs")
# 或按性别比较能力均值：
tapply(raw$ability, raw$gender, mean)
```

本章你应该掌握的：① `m$ksi` 的 `ksi` 是难度（大 = 难），`se.ksi` 是估计精度；② 区分度大 = 好题，负数 = 反向题没反转；③ `m$person$EAP` 是每个人的科学能力分，`SD.EAP` 是其精度；④ `m$variance/m$gmean` 看（各组）能力的离散和均值；⑤ AIC/BIC 越小越好、`anova` 比较嵌套模型；⑥ 加速后看 `accuracy_report$met` 是否为 TRUE；⑦ 用 `SD.EAP` 和边际信度判断问卷准不准；⑧ 综合区分度/难度/标准误判断好题坏题；⑨ 把 EAP 接回原始数据做后续分析。

完整实战案例（从文件到结论）

每个案例都是「从一份数据文件，到交付物」的完整流程。把你自己的文件名套进去即可。

案例 A：一份 0/1 的能力测验（最常见）

场景：500 个学生做了 30 道选择题（对 =1，错 =0），存在 `test.csv`。想知道：题目质量如何？每个学生水平多高？

```
library(IRTC)

mod <- irtc("test.csv", model = "2PL")      # 读取 + 清洗 + 体检 + 估计 + 评级，一行完成
plain_summary(mod)                          # 30 秒了解全局
mod$usability$quality                       # 逐题评级：哪些题「建议修改」？

irtc_excel(mod, dir = "结果")                # 三个 Excel：质量表 / 参数表 / 能力值表
irtc_report(mod, "教师报告.docx", audience = "survey")
irtc_report(mod, "校长摘要.docx", audience = "decision")
```

想比较 Rasch 和 2PL 谁更合适（统计路径）：

```
m1 <- irtc("test.csv", model = "1PL", verbose = FALSE)
m2 <- irtc("test.csv", model = "2PL", verbose = FALSE)
anova(m1, m2)      # 似然比检验；AIC/BIC 更小者更优
```

能写进报告的结论示例：「2PL 的 AIC 显著低于 Rasch ($p < 0.001$)，说明各题区分度确有差异。质量表显示第 7、19 题区分度低于 0.15，已标记为『建议修改』。学生能力值见能力值表，可直接并入总成绩册。」

案例 B：原始 A/B/C/D 作答 + 答案键

场景：导出的答题卡数据是选项字母，第 1 题正确答案 A，第 2 题 C……

```
key <- c(Q1="A", Q2="C", Q3="B", Q4="D", Q5="A") # 答案键
mod <- irtc("答题卡.xlsx", model = "2PL", key = key)
mod$usability$quality # 若某题「区分度为负——请排查!」, 优先怀疑答案键错误
```

案例 C: 5 点 Likert 满意度问卷（含反向题）

场景：300 人做 10 道满意度题（1~5 分），第 4、9 题反向，存在 `survey.sav`（SPSS 文件）。

```
d <- irtc_read("survey.sav") # SPSS 直接读; 1~5 自动重编码为 0~4
d$resp[, c(4, 9)] <- 4 - d$resp[, c(4, 9)] # 反向题仍需人工反转 (唯一要自己做的事)

mod <- irtc(d, model = "GPCM")
mod$usability$quality # 验证: 没有负区分度 = 反转处理正确
irtc_excel(mod, dir = "结果")
```

案例 D: 多维（两个分量表）

场景：16 题问卷，前 8 题测「焦虑」、后 8 题测「抑郁」，二分计分。

```
Q <- cbind(c(rep(1,8), rep(0,8)),
           c(rep(0,8), rep(1,8)))
mod <- irtc("twoscale.csv", model = "2PL", Q = Q) # Q 透传给估计引擎

cov2cor(mod$variance) # 两维相关: 如 0.62 = 中度相关但可区分
mod$person # 每人两个维度的 EAP 分数
```

案例 E: 大数据 + 受控精度加速 + 误差核查

场景：5 万人、60 题、3 维 GPCM，精确模式太慢，想加速但要确保可信。

```
mod <- irtc("big.csv", model = "GPCM", Q = Q,
           method = "auto",
           control = list(fast = TRUE, mass_budget = 1e-3,
                          verify = "stratified", verify_n = 3000,
                          refine = TRUE))

mod$routing # 看自动选了哪个引擎、为什么
mod$accuracy_report$met # TRUE = 实测误差在容差内, 结果可放心用
```

结论：若 `met==TRUE`，可写「采用受控精度流式估计，分层核查确认能力分误差远小于估计本身的不确定性，结果可靠」。

案例 F：分组比较（如男女差异）

```
raw    <- irtc_read("全表.xlsx", id = "学号")$resp
gender <- raw[["性别"]]
resp   <- raw[, setdiff(names(raw), "性别")]

mod <- irtc(resp, model = "2PL", Q = Q, group = gender)
mod$gmean      # 各组平均能力（参照组为 0）
mod$variance    # 各组协方差
```

结论示例：「以男生为参照组，女生的平均能力高 0.18 个标准单位；两组能力离散度相近。」

案例 G：跨年度题库链接（参数表的用法）

场景：今年和去年的测验共用 10 道锚题，想把两年成绩放到同一把尺子上。

```
irtc_excel(mod2025, dir = "2025", prefix = "Y2025")
irtc_excel(mod2026, dir = "2026", prefix = "Y2026")
# 两个 *_item_parameters.xlsx 列结构完全相同 (schema v1.1):
# 锚题保持相同 item_id, 按 item_id 合并即可对比 difficulty_b 做链接等值
```

多维模型请注意（1.1.2 修正）：1.1.2 之前，多维模型导出的参数表只从第 1 个维度读取区分度，凡是不载荷在第 1 维上的题目一律写成 `slope_a = 0`、`difficulty_b = NA`——而且 streaming 引擎导出的 `item_id` 还是 `I1..In` 这样的通用名，`p_value` 全为 `NA`。`item_id` 恰恰是链接等值的连接键。

1.1.2 起，参数按该题实际载荷的维度读取，并新增两列：

- `dimension`：这道题属于哪个维度（用你 Q 矩阵里的维度名）；
- `n_loadings`：这道题载荷在几个维度上。简单结构下恒为 1；若大于 1（一题跨多维），`slope_a` 是各载荷之和，属于复合量而非单一区分度，等值时要另行处理。

用 1.1.2 之前的版本从多维模型导出过参数表的，必须重新生成后再用于等值。

本章你应该掌握的：每类场景的骨架都是同一条流水线——`irtc(文件, model=, ...)` → `plain_summary/quality` → `irtc_excel/irtc_report`；多维加 `Q=`，分组加 `group=`，大数据加 `control=list(fast=TRUE, verify=...)`，原始作答加 `key=`。

数学定义与原理（为没学过统计的人解释）

这一章把背后的数学用大白话讲一遍。看不懂公式没关系，先看每段开头的「人话解释」。读完你会明白软件「到底在算什么」，从而更会用、更会判断结果。

7.1 概率：先统一一个最基本的词

人话：概率就是「发生的可能性」，从 0（绝不发生）到 1（一定发生）。0.5 就是一半一半。IRT 里反复出现的「答对的概率」，就是「这个人在这道题上答对的可能性有多大」。

7.2 逻辑斯蒂曲线：IRT 的「S 形」核心

人话：我们需要一个公式，能把「能力 θ 减去难度 b 」这个可正可负的数，变成一个 0 到 1 之间的「概率」。最常用的就是逻辑斯蒂函数，它画出来是一条平滑的 S 形曲线：

- 当能力远低于难度（ $\theta - b$ 很负）→ 概率接近 0（几乎答不对）；
- 当能力等于难度（ $\theta - b = 0$ ）→ 概率正好 0.5；
- 当能力远高于难度（ $\theta - b$ 很正）→ 概率接近 1（几乎一定对）。

公式（Rasch）：

$$P(\text{答对}) = 1 / (1 + e^{-(\theta - b)})$$

这里 e 是数学常数（约 2.718）， e^x 是指数函数。你不需要会算，只要记住这条 S 曲线的形状和上面三句话。

2PL 在指数里多乘一个区分度 a ： $P = 1 / (1 + e^{-a(\theta - b)})$ 。 a 越大，S 曲线越「陡」——能力稍微变化，概率就剧烈变化，所以「区分力强」。

多级题（GPCM）用类似的指数形式，给每个相邻档次之间算一个跨越概率，再归一化成各档的概率（细节见英文版/帮助页，原理一致）。

7.3 似然：「哪套参数最像我看到的数据？」

人话：假设我们先猜一组题目参数和能力。用这组猜测，我们能算出「在这组猜测下，恰好出现我手里这份真实数据的可能性」有多大——这个可能性就叫似然 (likelihood)。

- 如果某组猜测算出的似然很大，说明这组猜测「很能解释」我看到的數據——它「很像真的」。
- 估计的目标，就是找出让似然最大的那组参数——这叫极大似然估计。

打个比方：你看到地上一串脚印（数据），你猜「是谁走过的」（参数）。似然就是「如果真是张三走的，会留下这串脚印的可能性」。你会选那个最能解释脚印的人。

7.4 把能力「积分掉」：边际似然 (MML)

人话：每个人的能力 θ 我们其实不知道（它正是要估的）。在估计题目参数时，我们不想被「不知道的 θ 」卡住，于是用一个数学技巧：把所有可能的 θ 按其出现的概率「加权平均」掉——这叫对 θ 积分，得到的结果叫边际似然。

- 我们假设全体人的能力服从一个正态分布（钟形曲线，多数人能力中等、少数人很高或很低）。
- 「积分掉 θ 」=「考虑了每种能力值出现的可能性后，综合算出这道题数据的似然」。
- 这种方法叫 边际极大似然 (Marginal Maximum Likelihood, MML)，是 IRTC 的核心算法。

7.5 EM 算法：先猜能力、再修题目，来回迭代

人话：MML 的最大化不能一步到位，于是用一个「来回迭代」的聪明办法，叫 EM 算法：

1. **E 步（期望）**：在当前的题目参数下，算出「每个人最可能的能力分布」，并据此算出「每道题、每个档次大概被多少（加权的）人选了」——这叫期望计数。
2. **M 步（最大化）**：把这些期望计数当成「已知」，去更新题目参数和能力分布的方差，让似然更大。
3. 回到第 1 步，用新参数再算……如此反复，直到参数几乎不再变化（收敛）。

就像「先根据现有题目猜每个人的水平，再根据猜出的水平修正题目，再用修正后的题目重新猜水平……」直到稳定。IRTC 在这里用了很讲究的数值方法（解析梯度、牛顿法、必要时退回更稳的算法）保证又快又稳，还可选 **SQUAREM** 技巧加速收敛。

7.6 多维与「维度灾难」

人话：单维时， θ 是一个数，积分就是在一条线上取很多点求和。多维时 (D 个维度)， θ 是一个 D 维向量，积分要在一个 D 维的网格上求和——节点数是「每维点数」的 D 次方 (Q^D)。

维度一多，节点数指数爆炸：3 维 21 点就有 9261 个节点，4 维就有 19 万个。这叫维度灾难，是多维 IRT 慢的根源。

7.7 流式引擎：为什么能算百万人

人话：IRTC 的「流式引擎」用两招对付大数据：

1. 按维分解：当每道题只属于一个维度（简单结构）时，似然可以拆成各维分别算，先算便宜的「每维似然」，再在大网格上拼起来。
2. 逐人流式累加：不把所有人的中间结果都存在内存里，而是算一个人、累加一个人、丢一个人，所以内存占用几乎和人数无关——这就是百万人也不爆内存的原因。

软件还会预测「网格引擎」和「流式引擎」哪个更快，自动选 (`method="auto"`)。

7.8 受控精度：用「预算」换速度，用「核查」保安全

人话：即使有流式引擎，超大网格仍可能慢。于是提供近似加速：把那些「几乎没人会落在上面、对结果几乎没贡献」的网格节点剪掉，只保留重要的，从而提速。

- **mass_budget** (质量预算 ϵ)：你允许剪掉多少「先验质量」。但它只是预算，不直接等于结果误差。
- 为了不误伤极端被试（他们的能力落在罕见区域，那里先验小但对他们很重要），剪枝同时考虑「样本里实际有人落在哪」（后验占用），并对每个小组分别保护，再合并。
- 真正的安全保证来自实测核查：算完后，软件抽一个有代表性的子样本（含普通人、小组、高权重者、极端答题者），分别用「完整网格」和「剪枝网格」算一遍，对比两者差多少，报告误差的中位数/95% 分位/最大值，并判定是否在你设的容差内 (`met`)。这就把「近似」变成了「带实测误差报告的近似」，让你能放心。

7.9 模型识别：为什么要「固定一些东西」

人话：有些东西天生「分不开」。比如「把所有能力 +1、同时把所有题难度 +1」，作答概率完全不变——数据无法区分这两种情形。为了让答案唯一，必须人为固定一个基准（叫「识别约定」）：

- 尺度：把能力的方差固定为 1（相当于规定「一个标准单位有多长」）。
- 位置：把某个基准（如参照组的均值）固定为 0（规定「零点在哪」）。
- 多组：题目参数跨组共享，作为把各组放到同一把尺子上的「锚」。

这些固定不影响你关心的相对结论（谁难谁易、谁高谁低），只是定下了「尺子的零点和单位」。

7.10 信息准则：怎么公平比较模型

人话：复杂模型（参数多）总能把数据拟合得更好，但可能「过拟合」（把噪声也学了）。AIC、BIC 在「拟合好」和「别太复杂」之间打分：分越低越好。BIC 对复杂度罚得更重，样本大时更稳。带权重时，用「有效样本量」（Kish 公式）代替人数。

本章你应该掌握的：① 逻辑斯蒂 S 曲线把「能力 - 难度」变成概率；② 似然 = 「这套参数有多像真数据」，估计 = 找最大似然的参数；③ MML 把未知能力积分掉；④ EM 来回迭代「猜能力 ↔ 修题目」；⑤ 流式引擎靠「按维分解 + 逐人流式」省内存省时间；⑥ 加速用预算换速度、用实测核查保安全；⑦ 识别约定只是定下尺子的零点和单位。

全量报错解决手册

先看这里：易用层的每个报错都带错误码（形如 [\[E305\]](#)）、原因和「建议」三部分，照着建议做即可。错误码分段速查：

错误码段	属于哪个环节	典型例子
E001	缺可选依赖包	提示 <code>install.packages("openxlsx")</code> 后重试
E1xx	读取文件	E103 找不到文件；E105 不支持的扩展名
E2xx	答案键/计分	E204 答案键里的题名在数据中不存在
E3xx	数据体检	E305 有文字列（需给 <code>key=</code> ）；E308 有负数取值
E4xx	估计	E406 没写 <code>model=</code> ；E407 体检未通过（附清单）；E408 估计失败；E409 网格法内存不够
E5xx	导出/报告	E501 文件已存在（加 <code>overwrite=TRUE</code> ）

程序化处理：`tryCatch(表达式, irtc_error = function(e) e$code)`。

1.1.2 新增的错误码与警告码

码	何时出现	怎么办
W205	答案键里的某个答案在该题的实际作答中从未出现	核对答案键与数据文件里的选项编码；答案键按数据文件自身的编码写（见 3.6）

码	何时出现	怎么办
W427	维度间潜在相关被估计到边界 1.000	这通常说明各维度分不开、模型已退化为更低维；建议改用更少的维度重估并比较。此时的信息准则与 EAP 信度要谨慎解读
E409	显式指定 <code>method="grid"</code> ，但预计内存不够	报错里会写明预计需要多少 GB。改用 <code>method="streaming"</code> ，或减少积分节点数 <code>control=list(nodes=seq(-6,6,length.out=15))</code> 。 <code>method="auto"</code> 会自动避开这种情况
E410	<code>weights=</code> 长度与数据行数不符，或含负数/缺失	每个样本一个权重，行序一致，且必须为有限的非负数
E411	<code>irtc_audit_scoring()</code> 的输入类型不对	传入模型对象、 <code>irtc_data</code> 、 <code>irtc_results()</code> 结果或清洗日志 <code>data.frame</code>

以下各节处理估计引擎层的传统报错（多见于直接调用专家函数、跳过了自动清洗时）。

报错不可怕。每条都告诉你「你会看到什么 / 为什么 / 一步步怎么修」。先在本章用关键词搜你看到的英文信息。

8.1 数据与参数类

`pweights must be non-negative, finite, length N, not all zero.`

- 含义：你给的个案权重 `pweights` 不合法。
- 原因：权重长度不等于人数，或有负数 / NA，或全是 0。
- 解决：检查 `length(pweights)` 是否等于行数；`any(pweights<0)`、`any(is.na(pweights))` 是否为真；`sum(pweights)` 是否 >0。修正后再跑。

`Y contains missing values; resolve them before estimation.`

- 含义：你的协变量矩阵 `Y` 里有缺失值 NA。
- 原因：背景变量（年龄、教育等）有空缺。软件不会替你猜，要求你先处理。
- 解决：删除有缺失的个案，或用合理方法插补，使 `Y` 无 NA。

`group contains missing values.../pweights contains missing values...`

- 同上：分组向量 / 权重里有 NA。补全或删除这些个案。

extreme case weights (max/mean > 50). (警告，不是错误)

- 含义：权重极端不均，少数人的权重是平均的 50 倍以上。
- 风险：极少数高权重个体可能主导整个结果。
- 解决：核对权重是否正确（是不是用了推及人口的原始权数而没有归一化?）；必要时对极端权重做「截断（winsorize）」。结果仍会给出，但要谨慎解读。全国性调查中 max/mean 达到一百多是常见的，本身不一定是错误。

collinear covariates (columns ...) are not identified. (警告)

- 含义：协变量 Y 里有「共线」的列（比如两列完全一样，或一列能被其他列线性表示）。
- 解决：删掉冗余的协变量列。被点名的列其系数会被置 0。

8.2 引擎与路由类

1.1.2 起，作答数据里的 NA 不再是问题。1.1.1 及更早版本中，多维 streaming 引擎一旦遇到任何一个 NA 就会让 R 进程崩溃（caught segfault）。现在两个引擎都把缺失作答按「随机缺失」处理：该题对这个人不贡献信息，也不必删除任何个案。不需要再用 `complete.cases()` 预先剔除缺失。

method='streaming' unsupported for this model (within-item / non-simple structure); use method='grid'.

- 含义：你强制用了流式引擎，但你的模型它不支持（题内多维 / 非简单结构）。
- 解决：改用 `method = "grid"`；或把 Q 矩阵改成简单结构（每题只属于一个维度）。

Multiple group estimation is not (yet) supported for ... (来自 grid 引擎)

- 含义：网格引擎不支持多维 + 多组一起做。
- 解决：改用流式引擎 `method = "streaming"`（它支持多维多组）；或把模型降为单维。

accuracy verification did not meet tolerance; see accuracy_report. (警告)

- 含义：你开了加速（`fast=TRUE`），但实测核查发现误差超过了容差。
- 解决：① 把 `mass_budget` 调小（如从 `1e-3` 改 `1e-4`）；② 设 `refine = TRUE` 让它自动收紧重算；③ 看 `m$accuracy_report` 是哪个指标、哪个分位超标，针对性处理。

fast mode kept >70% of nodes; speed benefit is limited. (提示信息)

- 含义：在你的数据上，剪枝省不了多少节点，加速收益很小却仍承担近似风险。
- 解决：干脆关掉 `fast`（用默认的精确模式）。

8.3 数值与收敛类

现象：迭代很久不停 / 提示达到 `maxiter` 仍未收敛。

- 原因与解决：① 把 `control$maxiter` 调大（如 2000）；② 开 `control$acceleration="squarem"` 加速收敛；③ 若数据信息不足（题太少、人太少、维度太高），模型「弱可识别」，应简化模型或增加题目/样本。

现象：协方差矩阵奇异 / 非正定 / 报错。

- 原因：能力分布的协方差估计退化（常见于小样本、某维无题、共线）。
- 解决：① 加 `control$ridge = 1e-4`（小岭正则）；② 多组小样本组用 `reg = list(sigma_shrink_pooled = 0.3)` 向总体收缩；③ 检查是否某个维度没有任何题载荷（Q 矩阵某列全 0）。

现象：某题区分度 `a` 估计成很大的值（跑飞）。

- 原因：该题几乎完美区分（「数据分离」），似然在边界，估计不稳。
- 解决：用 `reg = list(slope_max = 4)` 设上限，或 `slope_penalty` 加惩罚（这属于流式的「统计正则」，会改变目标函数，软件会标注）。

现象：某道多级题结果异常 / 某类别从没人选。

- 原因：GPCM 里某个类别零计数，它的阈值无法识别。
- 解决：把这个空类别与相邻类别合并（重新编码），或检查计分是否有误。流式引擎有数值护栏，一般不会崩，但结果该类别不可信。

现象：多维能力对「真值」的相关只有中等（比如 0.5），怀疑算错。

- 解释：这通常是数据本身的可识别性问题，不是软件 **bug**（用标准网格 / 其他软件也是这个结果）。判断软件对不对的标准是「流式引擎 $\approx$ 网格引擎」，而不是「贴近你模拟的真值」。

现象：同样的数据，换了线程数，结果有约 10^{-8} 的微小差异。

- 解释：流式引擎把人分块给多个线程并行，再汇总，浮点求和的顺序取决于线程数，导致极微小（可忽略）的差异。要逐位复现就固定 `control$n_threads`；要「跨线程也完全一致」就用 `method="grid"`。这对任何实际结论都无影响。

8.4 安装与编译类

现象：`install.packages(...)` 时编译报错。

- 原因：在用源码安装，而电脑缺 C++ 编译工具链。
- 解决（首选）：改用 CRAN 的二进制包——运行 `install.packages("IRTC")`，遇到「是否从源码安装」的提问回答 `no`（Windows/macOS 不需要编译）。

- 解决（必须编译时）：Windows 装 **Rtools**；Mac 终端运行 `xcode-select --install`；Linux 装 `r-base-dev`。装好后重新安装 IRTC。

现象：报 **C++17 / CXX_STD** 相关错误。

- 解决：升级 R (≥ 3.5) 和编译器；本包需要支持 C++17 的编译器。

现象：**there is no package called 'RcppArmadillo' / 'Rcpp'**。

- 解决：先 `install.packages(c("Rcpp","RcppArmadillo"))`，再装 IRTC。

现象：**there is no package called 'IRTC'**（在 `library(IRTC)` 时）。

- 原因：IRTC 没装成功。
- 解决：回到第 2.6 节，先试 `install.packages("IRTC")`；若用的是本地源码包，确认 `install.packages("../tar.gz", repos=NULL, type="source")` 没报错。

现象：**package 'IRTC' is not available (for R version x.y.z)**。

- 原因：R 版本过旧（IRTC 需要 $R \geq 3.5.0$ ），或当前设置的 CRAN 镜像还没同步。
- 解决：升级 R；或运行 `chooseCRANmirror()` 换一个镜像（如 <https://cloud.r-project.org>）后重试。

现象：装好了，但 `packageVersion("IRTC")` 显示的不是 **1.1.2**。

- 原因：电脑上装了旧版本，或安装到了另一个库路径。
- 解决：运行 `update.packages(oldPkgs = "IRTC", ask = FALSE)`；仍不对则 `remove.packages("IRTC")` 后重新 `install.packages("IRTC")`。

现象：**cannot open file ... / 找不到文件**。

- 原因：`read.csv` 的路径写错了。
- 解决：核对路径（用 / 不用 \）、文件名、扩展名；或用 RStudio 的 Import Dataset 图形界面。

本章你应该掌握的：遇到任何报错，先复制其中的英文关键词，在本章搜索，按「原因—解决」一步步处理。绝大多数问题源于数据格式（第 3 章）或缺工具链（第 2.6 节）。

术语表（中英对照）

中文	英文	一句话解释
项目反应理论	Item Response Theory (IRT)	用「作答概率」把人的能力和题的性质放在同一把尺子上的理论。
潜在能力 / 特质	latent ability / trait (θ , theta)	看不见、要被测量的东西（如数学水平、满意度）。
项目 / 题目	item	问卷或测验里的一道题。
作答矩阵	response matrix	「行 = 人、列 = 题」的作答数据表。
二分计分	dichotomous	只有两种结果，编码 0/1。
多级 / 多分计分	polytomous	多个有序等级，编码 0,1,2,...。
难度	difficulty (b) / intercept (ψ)	一道题有多难答对；大 = 难。
区分度	discrimination (a / slope)	一道题分辨能力高低的本事；大 = 好题，负数 = 反向题没反转。
阈值	threshold	多级题里从一档跨到下一档的难度。
维度	dimension	一份问卷测的一个独立特质。
Q 矩阵	Q-matrix	指明「每题属于哪个维度」的 0/1 表。
单维 / 多维	uni- / multidimensional	测一个 / 多个特质。
简单结构 / 题间多维	simple / between-item structure	每道题只属于一个维度。
Rasch / 1PL	Rasch / one-parameter logistic	只估难度、假设各题区分度相同的二分模型。
2PL	two-parameter logistic	估难度和区分度的二分模型。

中文	英文	一句话解释
PCM / GPCM	(generalized) partial credit model	多级题模型，GPCM 还估区分度。
RSM	rating scale model	各题阈值结构相同的多级模型（适合统一 Likert）。
边际极大似然	marginal maximum likelihood (MML)	把未知能力积分掉后求最大似然的估计方法。
EM 算法	EM algorithm	「猜能力 ↔ 修题目」来回迭代的求解办法。
似然	likelihood	「某套参数有多像我看到的真数据」。
收敛	convergence	迭代到参数几乎不再变化。
EAP	expected a posteriori	每个人能力的点估计（科学能力分）。
标准误	standard error (SE)	一个估计有多准；越小越准。
方差 / 协方差	variance / covariance	离散程度 / 两维一起变化的程度。
潜在回归	latent regression	用背景变量解释/预测能力。
多组	multiple groups	不同人群各自一套能力分布。
个案权重	case weights (pweights)	抽样调查里每个人的权重。
信息准则	AIC / BIC	比较模型优劣的分数，越小越好。
偏差	deviance	$-2 \times$ 对数似然，越小拟合越好。
网格 / 流式引擎	grid / streaming engine	标准全网格 / 大数据省内存的计算引擎。
受控精度	controlled accuracy	用预算换速度、用实测核查保安全的加速。
质量预算	mass budget (ϵ)	允许剪掉多少先验质量；是预算不是保证误差。
核查报告	accuracy report	加速后实测误差的报告， <code>met=TRUE</code> 表示可信。
R 包	R package	给 R 加功能的插件，IRTC 就是一个。

中文	英文	一句话解释
函数 / 参数	function / argument	一条命令 / 给命令的设置选项。
路径	path / file path	文件在电脑里的「地址」。

推荐参考文献与书目

入门教材（强烈推荐先读）

- de Ayala, R. J. (2009). *The Theory and Practice of Item Response Theory*. Guilford. (公认最友好的 IRT 入门)
- Embretson, S. E., & Reise, S. P. (2000). *Item Response Theory for Psychologists*. Erlbaum. (面向不擅数学的读者)

模型奠基文献

- Rasch, G. (1960). *Probabilistic Models for Some Intelligence and Attainment Tests*. (Rasch 模型)
- Birnbaum, A. (1968). Latent trait models. In Lord & Novick (Eds.), *Statistical Theories of Mental Test Scores*. (2PL/3PL)
- Masters, G. N. (1982). A Rasch model for partial credit scoring. *Psychometrika*, 47, 149–174. (PCM)
- Andrich, D. (1978). A rating formulation for ordered response categories. *Psychometrika*, 43, 561–573. (RSM)
- Muraki, E. (1992). A generalized partial credit model. *Applied Psychological Measurement*, 16, 159–176. (GPCM)

估计方法（MML / EM / 多维）

- Bock, R. D., & Aitkin, M. (1981). Marginal maximum likelihood estimation of item parameters. *Psychometrika*, 46, 443–459. (MML/EM 基石)
- Adams, R. J., Wilson, M., & Wang, W. (1997). The multidimensional random coefficients multinomial logit model. *Applied Psychological Measurement*, 21, 1–23. (IRTC 多维框架的来源)
- Gibbons, R. D., & Hedeker, D. (1992). Full-information item bifactor analysis. *Psychometrika*, 57, 423–436. (维度降维思想)
- Schilling, S., & Bock, R. D. (2005). High-dimensional MML item factor analysis by adaptive quadrature. *Psychometrika*, 70, 533–555.
- Varadhan, R., & Roland, C. (2008). Accelerating the convergence of EM (SQUAREM). *Scandinavian Journal of Statistics*, 35, 335–353.

软件

- Chalmers, R. P. (2012). mirt. *Journal of Statistical Software*, 48(6).

- Eddelbuettel, D., & François, R. (2011). Rcpp. *Journal of Statistical Software*, 40(8).

手册级参考

- van der Linden, W. J. (Ed.) (2016). *Handbook of Item Response Theory* (3 卷). CRC Press.

IRTC 的技术路线、理论基础与优化效果

本章简要说明 IRTC 为什么能处理大型多维数据。一般使用者不必理解底层细节，只需知道：软件在尽量保持统计结果不变的前提下，减少了重复计算和内存占用；只有主动开启 `fast=TRUE` 时，才使用经过误差核查的近似加速。

11.1 总体技术路线

IRTC 的优化分为三层：先用 C++ 多线程加速最耗时的 E 步；再利用“每道题只属于一个维度”的简单结构，把题目计算从完整多维网格中拆出来，并按数据块边算边汇总；最后提供可选的节点剪枝，用少量、经过核查的近似误差换取更大提速。

技术路线	理论基础	实际作用
C++ 多线程与循环重排	并行计算、缓存局部性	多个被试和题目同时计算，减少重复读取和中间复制。
维度分解	题间多维 IRT 的条件独立性	将主要计算量由约 $N \times I \times Q^D$ 降为 $N \times I \times Q + N \times D \times Q^D$ ，题目越多优势越明显。
分块流式 E 步	MML-EM、充分统计量	不保存全部人的完整后验矩阵，内存由随人数增长改为基本受块大小控制。
E 步与统计量融合	EM 算法	后验算完立即汇总题目计数、能力矩和 EAP，避免反复扫描大矩阵。
SQUAREM 与安全化 Newton	固定点加速、梯度/Hessian、线搜索	减少 EM 收敛时间，并提高坏初值或病态数据下的稳定性。
自动选择 <code>grid/streaming</code>	计算复杂度与代价模型	根据人数、题数、维度和内存预测选择更合适的引擎，并在 <code>\$routing</code> 中说明原因。

技术路线	理论基础	实际作用
协变量模式缓存	潜在回归、重复模式复用	相同背景模式的人共用一次先验网格计算，离散协变量几乎不增加额外成本。
受控精度剪枝	数值积分、先验与后验质量	<code>fast=TRUE</code> 时删除不重要节点，并用分层样本实测 EAP、矩、参数和偏差误差。

11.2 精确性与稳定性原则

IRTC 默认使用精确路径：`fast=FALSE` 时保留完整积分网格；小数据或不适合流式计算的模型仍可使用 `grid`。多线程按被试分块，尽量保证不同线程数下结果一致。阻尼、线搜索和正定修复只用于稳定求解，不主动改变统计目标；会改变估计结果的惩罚、参数边界和协方差收缩均需用户显式开启。

11.3 已达到的效果

在百万样本、多维 GPCM 原型测试中，流式引擎约为 99 秒/次迭代，峰值内存约 2.4 GB；传统完整后验方案在同类规模下可能需要 TB 级内存。流式结果与标准网格结果高度一致，后续版本已支持多组、潜在回归和个案权重，并通过自动化测试与 `R CMD check`。开启快速模式后，应以 `m$accuracy_report$met` 为准：`TRUE` 表示分层实测误差达到设定容差，`FALSE` 时应收紧 `mass_budget` 或重新使用精确模式。

本章你应该掌握的：① IRTC 的主要提速来自多线程、维度分解和流式计算；② 默认精确模式不使用近似剪枝；③ `fast=TRUE` 的可信度要看 `accuracy_report$met`；④ 大数据优化的核心效果是把原本难以装入内存的多维计算压缩到普通电脑可运行的范围。

常见问题解答 (FAQ)

按调查工作者最常问的顺序排列。先扫一眼，多半你的疑问已在其中。

Q1: 我完全不会编程，真的能用吗？ 能。主工作流就一行：`irtc(" 你的文件.xlsx", model="1PL")`，再加 `plain_summary(mod)` 和 `irtc_excel(mod)` 共三行，从 Excel 文件到成品交付。第 2、4、6 章手把手教，照做即可。

Q2: 我应该用 Rasch(1PL) 还是 2PL/GPCM?

- 想要最简单、最稳、不同人群可比、或样本较小 → 用 Rasch/PCM (`irtc.mml`)。
- 想让数据自己说话、允许各题区分度不同、判断哪些题区分力强 → 用 2PL/GPCM (`irtc.mml.2pl`)。
- 拿不准就两个都跑，用 `anova()` 或 AIC/BIC 比较（见 5.6）。

Q3: 样本量要多少？ 经验值：Rasch 类每道题至少 100~200 人较稳；2PL/GPCM 因为多估了区分度，最好 300~500 人以上；多维模型需要更多。样本太小时优先用 Rasch。

Q4: 缺失值多了怎么办？ 少量缺失（标成 NA）IRT 能自然处理（用每个人实际作答的题来估计）。但若某人几乎全缺，建议删除该个案；若某题缺失率极高，考虑删除该题。

Q5: Likert 一定要从 0 开始编码吗？ 模型内部是的，但 IRTC 会自动帮你重编码：1~5 直接交给 `irtc()`，它会自动变成 0~4 并记入清洗日志。只有直接调用专家函数 `irtc.mml()`（跳过自动清洗）时才需要手工 `dat <- dat - 1`。

Q6: 怎么判断结果「跑对了」？

- 没有报错、提示收敛；
- 区分度没有负数（有负数 = 反向题没处理）；
- 难度、阈值在合理范围（大致 ± 3 内、阈值有序）；
- 用加速时 `accuracy_report$met == TRUE`。

Q7: EAP 能力分能直接当「分数」用吗？ 能。它比原始总分更科学（考虑了题目难易）。它是类似标准分的量纲（均值约 0，正高负低）。如果要给非专业读者看，可线性变换到熟悉的量纲（如均值 500、标准差 100）：`score <- 500 + 100 * scale(m$person$EAP)`。

Q8: 两个维度该不该合成一个? 看 `cov2cor(m$variance)` 的相关: 相关很高 (如 >0.9) 可能本就是一个维度; 中等相关 ($0.4 \sim 0.8$) 说明确实是两个相关但可区分的维度, 分开报告更合适。

Q9: 数据很大、跑得很慢怎么办? 用 `method="auto"` (默认会自动选流式引擎), 并开 `control=list(fast=TRUE, refine=TRUE)`。务必看 `accuracy_report$met` 确认误差可接受。

Q10: Excel 表和报告能换成英文吗? 能。 `options(irtc.lang = "en")` 后再导出即可; 也可以在单次调用里加 `lang = "en"`。机器可读结果 (`irtc_results`) 的列名本来就是固定英文, 不受影响。

Q11: 报错了先看什么? 先看错误码和「建议」两行——每个报错都自带修复建议 (如 [E305] …建议: 请先计分 `irtc_score(data, key=...)`)。按建议做仍解决不了, 再查第 8 章。

Q10: `method="auto"` 选了 `grid`, 但我觉得 `streaming` 更快? 路由是按预测时间选的, 通常是对的。看 `m$routing$reason`。若你确有把握, 可 `method="streaming"` 强制。一般不必干预。

Q11: 报错了但我看不懂英文? 复制报错里的关键英文词, 去第 8 章搜索。绝大多数报错在那里有「原因 + 解决」。

Q12: 结果每次跑都一样吗? 精确模式 (`fast=FALSE`) 下, 固定数据与设置, 结果逐位可复现。加速模式下若固定 `verify_seed`, 核查报告也可复现; 估计本身的极微小线程级抖动 ($\sim 10^{-8}$) 对结论无影响。

Q13: 我能比较「我这次的题」和「去年那套题」吗? 能, 但需要等值 (`linking/equating`): 两套题要有共同的锚题, 并用 `xsi.fixed` 把锚题参数固定到同一尺度。这属于进阶专题, 建议先读教材 (第 10 章 de Ayala 第 12 章)。

Q14: 什么时候选择 IRTC, 什么时候选择其他 IRT 软件? IRTC 适合需要 MML、流式计算、多组、个案权重或受控精度核查的任务。其他 IRT 软件可能支持不同模型或工作流, 应根据模型范围、数据规模和报告要求选择; 在共同模型上, 正确设定后的结果通常应彼此接近。

Q15: 可以做 3PL (带猜测) 吗? 本包暂不直接支持 3PL。需要时可用 `mirt` 等包。

把结果写进报告的模板

提示: `irtc_report(mod, " 报告.docx", audience="decision"/"survey"/"stat")` 已能自动生成三种成品报告 (含图表)。本章模板适用于你要手写报告、或要把 IRTC 结果并入更大的报告框架时——数字可直接从 `plain_summary(mod)` 和三个 Excel 表里抄。

直接套用, 把方括号 [...] 换成你的数字。

13.1 方法部分 (Methods) 模板

「本研究采用项目反应理论 (IRT) 对 [N] 名被试在 [I] 道 [二分/K 级] 题上的作答进行分析。采用 [Rasch/2PL/GPCM] 模型, 经边际极大似然 (MML) 估计 (IRTC 包, 版本 1.1.2)。[多维: 依据理论将题目划分为 [D] 个维度 (见 Q 矩阵)。][反向题第 [...] 题已反向计分。] 模型在 [迭代次数] 次迭代后收敛。」

13.2 题目质量表 (建议放附表)

题号	难度 b	区分度 a	标准误	判断
1	-0.8	1.2	0.07	良好
...	...	...	...	...
7	0.1	0.21	0.10	区分力低, 建议修订

配一句: 「全部题目区分度为正, 介于 $[a_{\min}]$ - $[a_{\max}]$; 第 [...] 题区分度低于 0.3, 建议修订或删除。」

13.3 信度与精度

「量表的边际信度为 [rel]，平均测量标准误为 [mean SD.EAP]，表明 [优秀/可接受/偏低] 的测量精度。测量在中等能力区间最精确，对极端能力个体精度较低。」

13.4 人群比较 / 潜在回归

「以 [参照组] 为基准，[另一组] 的平均 [能力] 高/低 [Δ] 个标准单位。」或：「潜在回归显示，[协变量] 每增加一个单位，[能力] 平均变化 [β] 个标准单位（控制其他变量）。」

13.5 加速与可信度（若用了 fast）

「为处理大样本，采用受控精度流式估计（质量预算 ε = [mass_budget]）。分层实测核查 ([verify_n] 名分层抽样个体，含小组、高权重与极端作答者）确认能力估计的 95% 分位误差为 [eap p95]，远小于估计本身的不确定性，结果可靠。」

13.6 一句话写「为什么用 IRT 而不是总分」

「相比简单加总，IRT 将题目难度与被试能力置于同一量尺，估计不依赖于具体题目集合，并提供逐人测量精度，因而更适合本研究的 [比较/等值/精细测量] 目的。」

V1.1.2 全函数参考（速查 + 详解）

本章覆盖 IRTC 1.1.1 全部导出函数（与 1.1.0 一致，1.1.1 未改动任何函数签名）。先给速查表，再逐个讲清签名、关键参数和返回值。深入的参数细节以包内帮助页为准（在 R 中输入 `?函数名`）。

14.1 全函数速查表

层	函数	一句话用途
一站式	<code>irtc()</code>	文件/数据框 → 清洗 → 计分 → 预检 → 估计 → 质量评级，一步完成
数据层	<code>irtc_read()</code>	读 Excel/CSV/TSV/SPSS/Stata/SAS/数据框并自动清洗，带日志；支持抽样权重导入
数据层	<code>irtc_score()</code>	答案键 0/1 计分或规则表多级计分；答案键/规则可为文件， <code>partial_answer</code> 列即分部计分
数据层	<code>irtc_read_q()</code>	读入 Q 矩阵（题目 × 维度）表/文件/矩阵，含可选的分部计分/满分列，规范化维度名
数据层	<code>irtc_align_q()</code>	把 Q 矩阵与作答数据按题目对齐，报告不一致（warn/error）
数据层	<code>irtc_check_data()</code>	估计前数据体检，输出机器可读问题清单和修复建议

层	函数	一句话用途
数据层	<code>irtc_audit_scoring()</code>	判定一份存档分析是否受 1.1.2 之前的答案键缺陷影响, 不必重跑
统计层	<code>irtc_ctt()</code>	经典测量统计: 难度、题总相关、Cronbach's alpha (支持 <code>weights=</code>)
统计层	<code>irtc_itemfit()</code>	题目拟合 <code>infit/outfit</code> 均方及 <code>t</code> 统计量 (支持 <code>weights=</code>)
统计层	<code>irtc_quality()</code>	四级白话质量评级 (好/可用/需检查/建议修改) 附原因和建议 (支持 <code>weights=</code>)
估计层	<code>irtc.mml()</code>	专家 API: Rasch/1PL、PCM、PCM2、RSM (固定斜率模型)
估计层	<code>irtc.mml.2pl()</code>	专家 API: 2PL、GPCM (估计斜率模型), <code>grid/streaming/auto</code> 引擎
输出层	<code>plain_summary()</code>	分层白话摘要: 先结论后细节
输出层	<code>irtc_excel()</code>	一次生成三个 Excel: 质量表 / 参数链接表 / 能力值表
输出层	<code>irtc_param_table()</code>	参数链接表的数据框 (不依赖 <code>openxlsx</code>)
输出层	<code>irtc_person_table()</code>	能力值表的数据框 (不依赖 <code>openxlsx</code>)
输出层	<code>irtc_report()</code>	Word/HTML 报告, <code>decision/survey/stat</code> 三种受众版式
输出层	<code>plot()</code> (<code>plot.irtc</code>)	Wright 图、能力分布、质量汇总、ICC 曲线
AI 层	<code>irtc_results()</code>	结构冻结的 tidy 结果列表 (schema v1.2, 仅增列不改列)
AI 层	<code>irtc_json()</code>	同上内容导出 JSON

层	函数	一句话用途
设置	<code>irtc_lang()</code>	查看当前输出语言；用 <code>options(irtc.lang="en")</code> 切换
S3 方法	<code>summary()</code> <code>print()</code> <code>logLik()</code> <code>anova()</code>	技术摘要、打印、对数似然、嵌套模型比较

14.2 一站式与数据层

```
irtc(data, model, key = NULL, rules = NULL, q = NULL, on_mismatch = c(
  "warn", "error"), rare_categories = c("collapse", "prior"), id = NULL,
  weights = NULL, sheet = 1, missing_codes = c(-9, -99, 99, 999), check =
  TRUE, quality = TRUE, verbose = TRUE, ...)
```

- **data**: 文件路径 (.xlsx/.xls/.csv/.tsv/.txt/.dat/.sav/.por/.dta/.sas7bdat/.xpt)、数据框、矩阵或 `irtc_read()` 的结果。
- **model**: 必填。"1PL" (同 "Rasch")、"2PL"、"PCM"、"PCM2"、"RSM"、"GPCM"，大小写不敏感。经验法则：对错题用 1PL/2PL，多级计分或量表题用 PCM/GPCM。
- **key/rules**: 原始作答的计分方式 (见 `irtc_score()`)；均可命名向量/数据框或文件路径；答案键文件加 `partial_answer` 列即分部计分。
- **q**: Q 矩阵 (题目 × 维度)，传给 `irtc_read_q()` 读取并与数据对齐；维度列名成为个人结果里的维度名 (`eap_< 维度 >/se_< 维度 >`)。
- **on_mismatch**: Q 矩阵与数据题目不一致时 "warn" (默认，保留共同题) 或 "error" (停止)。
- **rare_categories**: 没人得到的分数类别的处理，"collapse" (默认，并拢并记录映射) 或 "prior" (保留结构、稳定阈值)。
- **weights**: 抽样权重的列名或列号；默认自动识别常见权重列，作为 `pweights` 用于估计。
- **check**: 预检发现错误级问题时是否报错停止 (错误码 E407，附完整问题清单)。
- **...**: 全部透传给 `irtc.mml()/irtc.mml.2pl()`，专家参数不受限制 (如大写 Q 载荷矩阵、`group`、`pweights`、`Y`、`control`)。
- 返回: 标准 `irtc` 模型对象，外加 `$usability` (清洗日志、预检结果、CTT、拟合、质量评级、Q 对齐、权重、稀有类别信息)。无效题目 (全空、零方差) 自动剔除并告警 (W410)，并在 `irtc_results()` 里保留一行 (`status`)。

```
irtc_read(x, sheet = 1, id = NULL, weights = NULL, missing_codes = c(
  -9, -99, 99, 999), na_strings = ..., guess_id = TRUE, guess_weights =
  TRUE, clean = TRUE, recode = TRUE, verbose = TRUE)
```

- 自动识别分隔符 (逗号/制表符/分号/竖线) 和编码 (UTF-8/带 BOM/GBK)。

- **id**: 指定 ID 列 (名字或序号); 默认自动识别常见列名 (**id**、学号、姓名、准考证号等)。
- **weights**: 指定抽样权重列 (名字或列号)。**guess_weights=TRUE** (默认) 时自动识别常见权重列名 (**weight**、**wt**、**pweight**、权重、样本权重 等; 单字母 **w** 除外, 需显式指定); 权重须为正数, 缺失置 1 并告警。分离后存入 **\$weights**, 由 **irtc()** 作 **pweights**。
- **missing_codes**: 数值缺失码。负数一律转缺失; 正数只在明显超出该题作答范围时转缺失 (保护 0-100 计分中的合法 99 分)。
- **recode**: 把整数类别重编码为从 0 开始的连续分数 (如 1-5 量表 → 0-4), 逐题记入日志。
- 返回: **irtc_data** 对象 (**\$resp** 清洗后数据、**\$pid** 个案 ID、**\$weights** 权重、**\$log** 双语清洗日志)。

irtc_score(resp, key = NULL, rules = NULL, na_as_wrong = FALSE, sheet = 1)

- **key**: 命名向量如 **c(Q1="A", Q2="C")**, 或答案键文件路径 (列 **item/题目 + answer/答案**); 忽略大小写、空格和全角字符 (**A → A**)。未命名向量按列顺序匹配。文件里加 **partial_answer/部分正确答案** 列 (多个用 **|;、** 分隔) 即分部计分: 全对 =2、部分对 =1、其余 =0。
- **rules**: 数据框或文件 (**item, response, score**) 做多级计分; 没有规则的作答转缺失并告警 (W207)。
- **na_as_wrong = TRUE** 时缺失作答按 0 分计。
- **sheet**: 答案键/规则文件的 Excel 工作表 (序号或名称)。

irtc_read_q(x, sheet = 1)

- **x**: Q 矩阵的文件路径、数据框, 或带题目行名的数值矩阵。识别题目列 (**item/题目** 等); 其余列为维度, 列名即维度名; 可选的分部计分/满分列 (**partial/max_score/满分** 等) 声明多级题与满分。
- 返回: **irtc_qmatrix** 对象 (**\$Q** 载荷矩阵、**\$partial**、**\$max_score**、**\$log**)。

irtc_align_q(data, q, on_mismatch = c("warn","error"))

- 把 Q 矩阵 (**irtc_qmatrix** 或可被 **irtc_read_q()** 读取的对象) 与作答数据按题目名对齐并同序。只在一边出现的题目: **"warn"** 保留共同题并告警 (W420/W421), **"error"** 报错停止 (E422)。共同题少于 2 个时报错。

irtc_check_data(x, key = NULL, verbose = TRUE)

- 检查: 题目/样本数量、非数值列、负数与非整数取值、零方差题、极端缺失、稀疏类别、全空样本、重复 ID。
- 返回: **\$ok** (能否估计)、**\$issues** (数据框: **code**、**severity**、**where**、双语 **message** 与 **fix**)。AI 助手可直接按行处理。

irtc_audit_scoring(x)

判定一份存档分析是否受 1.1.2 之前的答案键缺陷影响 (见 3.6), 不需要重跑。

`x` 可以是: 估计好的 `irtc` 模型、`irtc_read()/irtc_score()` 的 `irtc_data` 对象、`irtc_results()` 的结果列表, 或只剩下的清洗日志 `data.frame` (导出的 Excel 工作簿和报告里都带着它)。

返回 `irtc_audit` 对象, `$status` 有五种取值: `not_scored` (没用答案键, 不可能受影响)、`not_affected` (用了但相关题目没重编号)、`ok_fixed` (1.1.2 及以后产出)、`affected` (确认受影响)、`affected_if_before_1.1.2` (条件命中但产出版本未记录)。`$affected_items` 列出需要重跑的题目, `$items` 给出逐题明细。

```
a <- irtc_audit_scoring(readRDS(" 模型对象.rds"))
a$status
a$affected_items
```

14.3 统计层

`irtc_ctt(x, key = NULL, weights = NULL)`

返回逐题 N、缺失率、均值、难度 (得分率)、校正题总相关、删题后 α , 及整卷 Cronbach's α 。

`irtc_itemfit(mod, resp = NULL, weights = NULL)`

基于 EAP 的残差拟合统计: outfit/infit 均方 (理想值 1.0, 常规范范围 0.7–1.3) 及 Wilson-Hilferty t 。streaming 引擎模型需通过 `resp=` 补传数据。

`irtc_quality(mod, resp = NULL, thresholds = NULL, weights = NULL)`

综合难度、区分度、拟合给出四级评级, 附双语原因与处理建议; 负区分度 (多为答案键错误) 直接判“建议修改”。阈值可用 `irtc_quality_thresholds()` 查看并通过 `thresholds=` 覆盖。

14.4 估计层 (专家 API, 与 0.1.0 完全兼容)

`irtc.mml(resp, irtmodel = "1PL", ...)` 与 `irtc.mml.2pl(resp, irtmodel = "2PL", method = c("auto", "grid", "streaming"), ...)`

完整支持: 潜在回归 ($Y=$ 或 $\text{formula}Y=/\text{data}Y=$)、多组 (`group=`)、抽样权重 (`pweights=`)、多维 Q 矩阵 (`Q=`)、参数固定与初值 (`xsi.fixed/xsi.inits/B.fixed/variance.fixed/beta.fixed`)、识别约束 (`constraint="cases"/"items"`)、斜率分组 (`est.slopegroups/E`)。control 列表含 `maxiter`、`conv`、`nodes`、`snodes` (随机节点, QMC= 选准蒙特卡洛)、`acceleration` (“Yu”/“Ramsay”)、`n_threads`、受控精度 `fast/mass_budget/verify` 等。详见第 4 章逐参数讲解。

14.5 输出层


```
plain_summary(mod, lang = irtc_lang())
```

打印五段式白话摘要：总体结论 → 分析概况 → 题目质量 → 能力分布 → 下一步；返回值可供程序复用。

```
irtc_excel(mod, dir = ".", prefix = "IRTC", lang = irtc_lang(), resp =
NULL, overwrite = FALSE, verbose = TRUE)
```

`irtc_param_table(mod, resp = NULL, weights = NULL)` 与 `irtc_person_table(mod, lang = irtc_lang(), weights = NULL)` 是这两张表的数据框版本，不需要 `openxlsx`。两者的 `resp` 与 `weights` 都会默认回退到模型自身保存的作答数据与权重。

生成三个独立.xlsx (需 `openxlsx`)：*_item_quality.xlsx (配色评级 + 白话建议 + 说明页)、*_item_parameters.xlsx (列结构冻结 schema v1.0, 跨年锚题按 `item_id` 直接合并)、*_person_ability.xlsx (行序与输入一致的平表)。已存在文件需 `overwrite=TRUE` (否则报 E501)。`irtc_param_table()/irtc_person_table()` 返回底层数据框，无需 `openxlsx`。

```
irtc_report(mod, file, format = NULL, audience = c("survey","decision",
"stat"), lang = irtc_lang(), ...)
```

按扩展名生成.html (自包含单文件，零额外依赖) 或.docx (需 `officer`)。decision 版 1-2 页给决策者；survey 版白话全文；stat 版含参数表、拟合表、模型信息与 ICC 曲线。

```
plot(mod, type = c("wright","ability","quality","icc"), lang = irtc_
lang(), items = NULL)
```

四种诊断图，与报告中嵌入的图一致；`type="icc"` 可用 `items=` 选题，仅限单维模型。

14.6 AI 与自动化层

```
irtc_results(mod, resp = NULL, weights = NULL) 与 irtc_json(mod, file =
NULL, resp = NULL, pretty = TRUE)
```

返回/导出五个结构冻结的数据框：`model_info` (模型与信息准则)、`items` (参数+CTT+拟合+评级)、`persons` (能力值表)、`cleaning_log`、`check_issues`。列名固定英文蛇形命名，与语言设置无关。完整 schema 见包内 `inst/llms.txt`。

结构化报错

易用层所有错误/警告都带 `$code` (如 E305)、`$reason`、`$fix`、`$data` 字段，并按域分类 (数据格式/计分/校验/估计/导出)。程序化捕获：`tryCatch(表达式, irtc_error = function(e) e$code)`。错误码分段：E0xx 依赖缺失、E1xx 读取、E2xx 计分、E3xx 校验、E4xx 估计、E5xx 导出。

`irtc_lang()`

返回当前输出语言 (`"zh"/"en"`)。全局切换: `options(irtc.lang = "en")`。机器可读结果不受语言影响。

附录

附录 A：一页速查卡

```

# ---- 安装与加载（首次一次，之后每次开 R 只需 library） ----
install.packages("IRTC")                # CRAN 正式版 1.1.2
library(IRTC)
packageVersion("IRTC")                  # 应为 '1.1.2'

# ---- 主工作流（一行到交付） ----
mod <- irtc("数据.xlsx", model = "2PL")  # 读取 + 清洗 + 体检 + 估计 + 评级
mod <- irtc("答题卡.csv", model = "1PL",
           key = c(Q1="A", Q2="C"))      # 原始 A/B/C/D 用答案键
plain_summary(mod)                      # 白话摘要
mod$usability$quality                   # 每题评级
irtc_excel(mod, dir = "结果")            # 三个交付 Excel
irtc_report(mod, "报告.docx", audience = "decision") # 成品报告
plot(mod, type = "wright")              # 诊断图

# ---- 存档核查（用过 key= 计分的旧分析） ----
irtc_audit_scoring(readRDS("模型对象.rds"))$status

# ---- 数据工具（按需） ----
d <- irtc_read("数据.sav")               # 六类格式直读 + 清洗
chk <- irtc_check_data(d); chk$issues    # 一键体检
d$resp[, c(4,9)] <- 4 - d$resp[, c(4,9)] # 反向题反转（唯一要手做的）

# ---- 专家路径（整理好的数值矩阵） ----
m1 <- irtc.mml(resp = dat)               # Rasch/1PL
m2 <- irtc.mml.2pl(resp = dat, irtmodel = "2PL") # 2PL
Q <- cbind(c(1,1,0,0), c(0,0,1,1))      # 多维 Q 矩阵
m4 <- irtc(dat, model = "GPCM", Q = Q,    # 大数据加速 + 核查
           control = list(fast=TRUE, mass_budget=1e-3, refine=TRUE))
m4$accuracy_report$met                  # TRUE 才放心

# ---- 读结果 ----

```

```
summary(m2); m2$xsi; m2$person$EAP      # 技术输出
cov2cor(m2$variance); anova(m1, m2)    # 维度相关; 模型比较
irtc_results(mod); irtc_json(mod, "r.json") # 机器可读 (AI/流水线)
options(irtc.lang = "en")               # 输出切英文
```

附录 B: CSV 数据模板 (二分题)

把你的数据存成这样的 CSV (第一行是题名, 之后每行一个人):

```
item1,item2,item3,item4
1,0,1,1
1,1,0,1
0,0,1,0
```

多级题同理, 格子放 0,1,2,... (记得从 0 开始)。

附录 C: 新手常用 R 基础操作

```
x <- 5          # 赋值: 把 5 存进 x
v <- c(1, 2, 3) # 造一个向量
m <- matrix(0, 3, 4) # 造一个 3 行 4 列、全 0 的矩阵
dat[1, ]        # 取第 1 行 (第 1 个人)
dat[, 2]        # 取第 2 列 (第 2 道题)
dat[, -(1:2)]   # 去掉前 2 列
nrow(dat); ncol(dat) # 行数、列数
mean(v); sd(v)  # 平均、标准差
?irtc.mml       # 打开某函数的帮助页 (最权威)
```

遇到不会的, 永远可以: 在控制台输入 `? 函数名` (如 `?irtc.mml.2pl`) 打开官方帮助页; 或回到本手册对应章节。

附录 D: 版本与更新记录

我该用哪个版本?

用 1.1.2。它修正了 1.1.1 及更早版本中三处会静默产生错误结果的缺陷。用 `install.packages("IRTC")` 装到的就是最新版; 用 `packageVersion("IRTC")` 可以随时核对。

1.1.2 (2026-08-27, 正确性修正版)

一次真实的全国性调查分析 (85,035 人、10 道二值题、三个内容维度、带抽样权重) 暴露了 1.1.1 的一批缺陷, 随后又用原始 SPSS 文件做全量验证, 共修正 15 处。

题目参数、潜在协方差、EAP 能力估计不变——变的是报告出来的拟合量、加权口径下的统计量，以及此前算错的那些输出。

会静默产生错误结果的三处（务必检查旧分析）

1. 答案键可能作用在错误的选项上。读入数据时选项 1..5 会被重编号为 0..4，而 1.1.1 的答案键是拿去和重编号后的值比对的。用 `irtc(key=)` 对从 1 开始编码的原始作答计分的分析，计分结果很可能整体错误。排查方法见 3.6 的 `irtc_audit_scoring()`，不必重跑即可判定。
2. 多维 streaming 引擎的对数似然与 AIC/BIC 有系统性偏移（等于 $n \times \text{维度数} \times \log(1/\text{节点步长})$ ）。同维度同设定下互比不受影响，但单维 vs 多维、或 grid 与 streaming 混用比较时完全失效。据此得出过维度数结论的，请重新比较（见 5.6）。
3. 多维模型的题目参数表只读第 1 维：不载荷在第 1 维的题目一律 `slope_a = 0`、`difficulty_b = NA`。用于跨年度等值的参数工作簿必须重新生成（见案例 G）。

其他修正

- 多维 streaming 引擎遇到任何一个 NA 会让 R 进程崩溃（段错误）；现在两个引擎都原生支持缺失作答，不必再预先剔除缺失个案。
- 潜在协方差退化（相关到达边界 1.000）时对数似然符号翻转、`EAP.rel` 明显偏低；现已修正，并新增 W427 警告。
- 多维 streaming 路径丢失 `id=`（退化成行号）、`pweight` 恒为 1；现均已保留。多维模型也开始给出能力值标准误。
- 抽样权重贯穿整条链路：经典难度、题总相关、alpha、Infit/Outfit、质量评级，以及个人的百分位与 T 分数，全部改为加权口径（见 4.1、5.4）。此前只有 IRT 参数是加权的。
- 个案 ID 若是「整十整百」的数字（如 2660000000）可能被写成科学计数法而破坏合并键；现已修正。
- streaming 模型此前不保存作答数据与题目名称，导出的参数表用 `I1..In` 通用名且 `p_value` 全空；现均已修正，多维模型调用 `irtc_quality()` 等也不再需要回传 `resp=`。
- 显式指定 `method="grid"` 而内存不足时，给出带处置建议的 E409，不再是 R 底层的内存错误。
- `irtc_read()` 的 ID 自动识别新增 `rid`、`respondent_id`、`record_id` 等列名。
- 新增 `irtc_audit_scoring()`；`irtc_results()` 结构版本升至 1.2、等值工作簿升至 1.1（均为纯增列）；两个引擎的 `ic` 统一为单行 data.frame。

升级后需要重做的事

情况	是否需要重做
用 <code>irtc(key=)</code> 对从 1 开始编码的原始作答计分	需要重跑（先用 <code>irtc_audit_scoring()</code> 判定）
依赖跨维度或跨引擎的 AIC/BIC 比较得出的结论	需要重新比较

情况	是否需要重做
从多维模型导出的题目参数工作簿	需要重新生成
加权分析产生的百分位 / T 分数常模	需要重新生成（数值会变）
加权分析的经典统计量与质量评级	数值会变，按新口径重新解读
不使用权重、单维、数据已计好分的分析	无需改动，结果逐位不变

1.1.1 (2026-07-17, CRAN 正式发布)

打包修正版：函数、参数和估计结果与 1.1.0 完全相同，升级不会改变你任何一次分析的数字。它的存在是为了通过 CRAN 的自动检查：

- 帮助文档（Rd 源文件）中进入 LaTeX 排版的内容改为纯 ASCII，官方 PDF 参考手册现在能正常生成。中文列名别名依然被完整支持和记录：新增的 `\zh` 宏让 HTML/文本帮助页照常显示中文字符，PDF 手册里显示等价的 `\uxxxx` 转义。
- DESCRIPTION 补上了 Date 字段，因此加载时的版本横幅是完整的 * IRTC 1.1.1 (2026-07-17)，而不是 * IRTC 1.1.1 ()。
- 一项内部测试不再假设 R 的固定措辞（开发版 R 会输出 R Under development (unstable)），改为与 R.version.string 比较。这只影响开发者的测试，不影响使用。

对你的实际影响：没有代码需要改。如果你按 1.1.0 的手册写过脚本，原样就能在 1.1.1 上跑出相同结果。

1.1.0 (易用性版本, 1.1.1 的功能基础)

估计核心 (`irtc.mml()` / `irtc.mml.2pl()`) 未变，新增功能都在易用层，向后兼容 1.0:

- `irtc_read(weights=)`: 抽样权重导入，并能自动识别常见的中英文权重列名；权重会被校验（必须为正数，缺失补 1 并给出警告），删除空行时保持对齐，`irtc()` 自动转发为 `pweights`。
- `irtc_read_q()` / `irtc_align_q()` / `irtc(q=)`: Q 矩阵（题目 × 维度）可从任意支持的文件格式或 R 对象读入，支持可选的分部计分/满分声明列；维度列名成为个人结果里的维度名；与作答数据不一致时可 `warn`（默认保留共同题）或 `error`。
- `key` / `rules` 可以直接写文件路径；答案键文件加 `partial_answer` 列即得到分部计分（全对 = 2、部分对 = 1、其他 = 0），并会与 Q 矩阵的分部计分声明做一致性检查。
- `irtc(rare_categories=)`: 处理没有人达到的分数类别。“collapse”（默认）合并未观测类别并记录映射，“prior”保留类别结构并稳定相关阈值；无人作答的题目会保留带标记的行，而不是悄悄消失。
- GPCM 题目输出带语义化的难度标签 (`b_partial` / `b_full`, 或 `b_step1..b_stepK`)；`irtc_results()` schema 升到 1.1（只增列，不改列）。
- `irtc_report()` 新增「模型诊断」（收敛、信息准则、EAP 信度、题目拟合解读）与「数据处理透明度」（权重、Q 对齐、类别合并、剔除题目、计分汇总、清洗日志）两节。

1.0.0 (首个易用层版本)

在 0.1.0 的估计核心之上增加面向四类人群的易用层：一站式 `irtc()`、`irtc_read()` / `irtc_score()` / `irtc_check_data()`、经典统计与题目拟合、四级质量评级、`plain_summary()`、三件套 Excel 导出、三种受众的 Word/HTML 报告、诊断图、机器可读结果 (`irtc_results()` / `irtc_json()`)、结构化报错和中英双语输出。新增依赖全部为可选 (Suggests)。

0.1.0 (估计核心)

Rasch/1PL、PCM、PCM2、RSM、2PL、GPCM 的 MML 估计，单维与题间多维、潜在回归、多组、个案权重、EAP 能力估计，并行 grid 引擎、有界内存 streaming 引擎，以及可选的受控精度模式（附实测误差报告）。

完整的英文更新日志见包内 `NEWS.md`，在 R 里也可以直接运行 `news(package = "IRTC")` 查看。

本手册对应 IRTC 1.1.2。函数签名以包内帮助页（如 `?irtc`、`?irtc.mml`、`?irtc_excel`、`?irtc_audit_scoring`）为准。如发现手册与软件行为不一致，以软件帮助页为准。

IRTC

项目反应理论分析工具包

Marginal Maximum Likelihood Estimation for Item Response Models

版本 1.1.1 (2026-07-17) · 已发布于 CRAN

制作人 / 维护人: 马崑翔 (Kunxiang Ma)

makunxiang@weandata.com

版权所有 © 2026 惟安数据科技 (北京) 有限公司

许可证 GPL (≥ 2)

项目主页 <https://github.com/weandata/IRTC>

CRAN <https://CRAN.R-project.org/package=IRTC>

问题反馈 <https://github.com/weandata/IRTC/issues>

本手册随软件版本更新; 如手册与包内帮助页 (`?irtc`) 不一致, 以帮助页为准。